

RESEARCH ARTICLE

Enhancing Smart Contract Security Through DevSecOps: An Adaptive Approach for Vulnerability Detection

NGHIA DINH^{ID}, VINH TRUONG HOANG^{ID}, BAY NGUYEN VAN^{ID}, THIEN HO HUONG^{ID},
HA DUONG THI HONG^{ID}, HAU NGUYEN TRUNG^{ID}, AND KIET TRAN TRUNG^{ID}

Faculty of Information Technology, Ho Chi Minh City Open University, Ho Chi Minh City 722000, Vietnam

Corresponding author: Vinh Truong Hoang (vinh.th@ou.edu.vn)

This work was supported in part by the Ho Chi Minh City (HCM) Open University, Vietnam.

ABSTRACT Smart contracts are central to decentralized applications but are often vulnerable to security flaws, both during development and in deployment. Due to their immutable nature, any vulnerabilities introduced are permanent and exploitable, highlighting the need for a secure, structured development lifecycle. This study proposes an adaptive DevSecOps approach tailored to blockchain applications, drawing on established practices and gray literature. By embedding security into Continuous Integration and Continuous Delivery (CI/CD), the model ensures continuous protection from development through deployment. At its core is an adaptive cognitive security framework that automates vulnerability detection using techniques like static analysis, fuzzing, and symbolic execution. It also streamlines secure provisioning across various environments. To further enhance security, Agentic AI is integrated into the pipeline. These autonomous agents monitor code, infrastructure, and behavior in real-time, learning from past incidents, detecting threats, applying patches, and adapting CI/CD workflows dynamically. Experimental results validate the model's effectiveness, showing significant improvements in the security and reliability of smart contracts. This approach offers a scalable, proactive solution for securing blockchain systems in a rapidly evolving threat landscape.

INDEX TERMS Security, blockchain, smart contract, DevSecOps, quantum security, agentic AI.

I. INTRODUCTION

Blockchain technology [1] has emerged as one of the most transformative innovations in recent years due to its inherent qualities of decentralization, immutability, and tamper-resistance. Fundamentally, a blockchain is a distributed, shared ledger of transactions that is maintained collectively by the member nodes of a network and governed by a consensus mechanism [2]. Among its most notable applications, smart contracts [3] have garnered substantial attention in both industry and academia.

The concept of smart contracts was first introduced by Nick Szabo in 1994 [4]. A smart contract is a self-executing code that runs on a blockchain, typically written in a Turing-complete language such as Solidity. These contracts

enable automated, transparent transactions without the need for centralized intermediaries. However, with the rapid increase in the deployment of smart contracts, numerous security vulnerabilities have been exposed, leading to significant financial losses. One striking example occurred in April 2018, when an attacker exploited an integer overflow vulnerability in the Beauty Chain (BEC) contract [5], allowing the unauthorized issuance of unlimited tokens and causing the token's value to crash to zero. Similarly, in May 2019, a major breach at Binance Exchange [6] resulted in the loss of over 7,000 BTC due to a coordinated cyberattack. These incidents highlight the urgent need for robust vulnerability detection and comprehensive security analysis in smart contract development.

The core challenge lies in the inherent difficulty of validating smart contracts, especially those written in complex programming languages. The interaction with external

The associate editor coordinating the review of this manuscript and approving it for publication was Sedat Akleyek^{ID}.

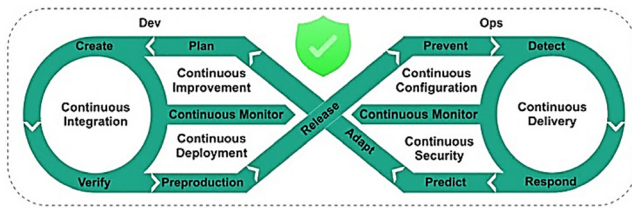


FIGURE 1. DevSecOps processes.

contract interfaces or third-party components increases the risk of reentrancy attacks, unexpected state changes, and unpredictable behavior. Compounding this is the immutable and irreversible nature of smart contracts once deployed, any discovered vulnerability can be difficult or impossible to patch. Security risks may arise not only during the development phase but also from configuration flaws, unsafe deployment environments, or inadequate testing processes. Consequently, there is a pressing need to embed smart contract development within a structured software development lifecycle (SDLC), ensuring rigorous testing, validation, and security assurance.

To address these challenges, the adoption of DevSecOps [7], as illustrated in Figure 1, offers a promising strategy for enhancing the security and reliability of blockchain applications. DevSecOps extends the principles of DevOps by embedding security as a shared responsibility throughout the entire development lifecycle.

Although DevOps emphasizes speed and automation to shorten development and deployment cycles, it often overlooks essential security practices, exposing systems to vulnerabilities introduced during rapid iteration. DevSecOps bridges this gap by integrating security checks, policies, and tools into every phase of the pipeline, from design and coding to deployment and monitoring.

An emerging enhancement to this paradigm is the integration of agentic AI, which introduces autonomous and intelligent agents capable of making real-time decisions, learning from system behavior, and proactively addressing security threats. These agents can dynamically adapt security policies, monitor the pipeline for anomalies, detect vulnerabilities as they emerge, and even trigger remediation workflows without human intervention. Their continuous learning capabilities allow them to evolve in parallel with the software, improving both accuracy and response time over traditional rule-based approaches.

Two essential components form the foundation of DevSecOps:

- Continuous Integration (CI): Enables regular code integration, automated builds, and testing to identify defects early.
- Continuous Delivery/Deployment (CD): Automates deployment processes to ensure consistent and secure software delivery.

The proposed adaptive secure strategy leverages these DevSecOps principles while embedding agentic AI into the

security fabric of the pipeline. This enhances the framework by enabling:

- Early and autonomous identification of vulnerabilities, reducing remediation cost and complexity.
- Intelligent, real-time decision-making that improves software quality and ensures secure contract logic before deployment.
- Self-orchestrated and secure deployments across heterogeneous environments, driven by adaptive AI monitoring and enforcement.

While this study focuses on smart contract development within the Ethereum ecosystem, the DevSecOps methodology, augmented by agentic AI, is platform-agnostic and can be extended to other blockchain platforms such as Binance Smart Chain, Polkadot, or Solana. This adaptability reinforces DevSecOps as a universal and intelligent security framework for the evolving landscape of decentralized applications.

The structure of this study is organized to provide a comprehensive exploration of smart contract security within the DevSecOps framework. Section II offers a detailed review of related work, positioning our study within the broader context of blockchain security research. Section III examines the core challenges and risks inherent in smart contract development, including common vulnerability types and their real-world implications. In Section IV, we analyze a range of vulnerability detection techniques, encompassing both traditional methods and emerging AI-driven approaches. Section V introduces the foundational principles of DevSecOps and discusses how they contribute to strengthening the security posture of blockchain-based systems, with a particular emphasis on smart contracts. Section VI delves into the integration of vulnerability detection mechanisms across critical DevSecOps phases, such as Continuous Integration, Continuous Deployment, and monitoring, highlighting best practices for embedding security into the development lifecycle. Section VII presents the experimental setup, including datasets, evaluation metrics, and an ablation study to assess the impact of various DevSecOps components on detection performance. Finally, Section VIII and IX offer a discussion of the findings, outlines potential areas for improvement, and concludes the study with key insights and directions for future work.

II. RELATED WORK

In recent years, an increasing number of scholarly contributions have addressed the security of smart contracts, focusing on a range of techniques including fuzz testing, symbolic execution, program analysis, formal verification, and machine learning. These approaches aim to identify and mitigate vulnerabilities in smart contract code before deployment, as the immutable nature of blockchain renders post-deployment fixes difficult or impossible.

One of the most foundational techniques is symbolic execution, which replaces concrete inputs with symbolic

representations to analyze all feasible execution paths within a program. By symbolically interpreting the code, constraints are derived for each conditional branch, enabling analysts to determine the inputs that trigger specific execution paths. This facilitates the detection of hidden vulnerabilities that may not be exposed through standard testing. Tools such as Oyente [8], Maian [9], Osiris [10], Securify [11], and Slither [12] have been developed based on this method to analyze Ethereum smart contracts.

In contrast, fuzz testing is a dynamic vulnerability detection method that involves generating random or semi-random inputs to test for unexpected behavior or crashes. By monitoring the system's responses, fuzzing identifies potential security issues caused by edge cases or malformed inputs. Tools such as ContractFuzzer [13] and Fuzzing [14] utilize this approach to assess smart contract robustness.

Formal verification represents one of the most rigorous techniques for validating smart contracts. By modeling contract logic in a formal language and applying mathematical proofs, it can ensure that the implementation adheres to its specifications with a high degree of confidence. Notable formal verification methods include the work of Hildenbrandt et al. [15] and VerX [16], which demonstrate strong guarantees of correctness for smart contract behaviors.

Program analysis, including taint analysis, is also extensively used to track data flow and identify the impact of untrusted inputs on critical contract operations. Taint analysis helps detect vulnerabilities like reentrancy, uninitialized storage, and access control flaws. Tools like Slither [12] and SmartCheck [17] offer rule-based and pattern-based detection mechanisms for static analysis of smart contracts.

In recent developments, machine learning (ML) has gained traction as a powerful method to automate vulnerability detection in smart contracts. By extracting control flow graphs and semantic features from contract code, ML models can learn to classify vulnerabilities and predict potential exploits. Research by Sendner et al. [18] and Russell et al. [19] showcases the effectiveness of AI techniques in identifying patterns that are difficult to capture through manual analysis or rule-based systems.

Despite the significant advancements in vulnerability detection, academic literature that specifically addresses the integration of DevSecOps with blockchain-based software remains sparse. However, as shown in Table 1, several studies examine the application of DevSecOps practices in blockchain environments, acknowledging the unique challenges these systems face. For instance, Li et al. [20] delve into the complexities of building and maintaining blockchain networks, emphasizing the crucial role of DevOps in overcoming operational barriers that can impede blockchain adoption. Koul [21] addresses the intricate process of testing blockchain-based applications, offering targeted strategies and tools designed to ensure quality assurance while maintaining system integrity. Yussupov et al. [22] explore the fusion of blockchain and smart contracts within serverless architectures, highlighting the importance

of DevOps in automating deployment processes and ensuring efficient development workflows. Other works, such as Yilmaz et al. [23], propose distributed event recording mechanisms that enhance transparency and traceability throughout the software development lifecycle, while Beller and Hejderup [24] suggest leveraging blockchain technology to build trusted, democratized package repositories aimed at addressing supply chain integrity issues.

The intersection of AI, ML, and DevSecOps in blockchain environments has also garnered attention. Fu et al. [25] examine how artificial intelligence and machine learning can enhance DevSecOps pipelines by providing predictive threat modeling, anomaly detection, and intelligent automation of vulnerability assessments, thereby improving the security of blockchain-based systems. Manikyala et al. [26] further contribute to this discourse by exploring the integration of cybersecurity measures within DevOps pipelines, with a focus on strengthening the security of distributed systems. Their study underscores the importance of early security integration, automated security testing, and the establishment of a security-first culture. Additionally, the authors advocate for continuous monitoring and incident response mechanisms to address potential vulnerabilities, suggesting adaptive cybersecurity frameworks that encourage collaboration between development, operations, and security teams. Furthermore, they call for regulatory guidelines tailored specifically to the unique challenges faced by DevOps in blockchain contexts to help protect sensitive data and maintain user trust.

In a related vein, Datadog [27] has developed a maturity model for assessing DevSecOps adoption, evaluating organizations based on their level of automation, the enforcement of security policies, and the cultural shift toward security by design. This model serves as a benchmark for organizations striving to integrate security practices into their DevOps workflows more effectively. Meanwhile, Santos et al. [28] investigate how DevSecOps practices can facilitate regulatory compliance within blockchain environments. They introduce tools for continuous compliance auditing, policy validation, and secure infrastructure provisioning, providing a framework for organizations to navigate the complexities of regulatory adherence while developing and deploying secure blockchain applications. This growing body of research highlights the importance of integrating DevSecOps into blockchain software development and calls for more comprehensive studies to address the evolving challenges of securing blockchain systems.

In this paper, we present an adaptive, cognitive, blockchain-oriented vulnerability detection approach by integrating advanced security tools into every stage of the DevSecOps pipeline. Our methodology ensures the seamless embedding of security within the Continuous Integration and Continuous Delivery (CI/CD) lifecycle, providing comprehensive coverage from the initial development phase to final deployment. Each phase of the DevSecOps process is tailored with specific configurations that activate the most relevant security tools and testing techniques in a

TABLE 1. DevSecOps research.

Research	Category	Description
Li et al. [20]	Industry Practices	Conducted an evidence-based study on DevSecOps in industrial settings. Highlighted that DevOps practices often trade security for speed, with security activities often treated as an afterthought. Emphasized the need for integrating security across the CI/CD lifecycle.
Koul [21]	Literature Review	Presented a multivocal literature review (2012–2021), introducing a Challenge-Practice-Tool-Metric (CPTM) model. Identified core aspects such as definitions, challenges, tools, and measurement strategies of DevSecOps.
Yussupov et al. [22]	Practice & Tool Mapping	Conducted a systematic literature review and classified 39 DevSecOps practices, including SAST, DAST, and threat modeling. Mapped tools and practices to each CI/CD stage to guide secure software development.
Yilmaz et al. [23]	Security Automation	Investigated security automation in DevOps pipelines using AI. Emphasized the need for shared ownership between development and security teams and highlighted trends like automated risk assessment and intelligent alerting.
Beller and Hejderup [24]	Integration Challenges	Discussed the integration of security tools and feedback loops in CI pipelines. Focused on the technical and organizational difficulties in aligning security and development workflows.
Fu et al. [25]	AI in DevSecOps	Explored how AI and ML enhance DevSecOps pipelines through predictive threat modeling, anomaly detection, and intelligent automation of vulnerability assessments.
Manikyala et al. [26]	Security Toolchain Integration	Explores how integrating cybersecurity into DevOps pipelines enhances distributed system security. It stresses the importance of early security integration, automated testing, and fostering a security-first culture. The study also highlights continuous monitoring and incident response to address vulnerabilities and advocates for adaptive frameworks promoting collaboration among development, operations, and security teams. The authors call for regulatory guidelines tailored to DevOps challenges to better protect data and maintain user trust.
Datadog [27]	DevSecOps Maturity Model	Developed a maturity model for DevSecOps adoption. Assessed organizations based on their automation level, security policy enforcement, and cultural shift toward security by design.
Santos et al. [28]	DevSecOps and Compliance	Studied how DevSecOps can support regulatory compliance. Introduced tools for continuous compliance auditing, policy validation, and secure infrastructure provisioning.

synchronized and coordinated manner. The integration of these security measures offers several significant advantages:

- **Early Detection of Vulnerabilities:** By incorporating security testing early in the development stages, smart contract vulnerabilities can be identified and addressed promptly, reducing the likelihood of costly issues post-deployment.
- **Continuous Quality and Security Assurance:** The approach maintains consistent security oversight throughout the smart contract lifecycle, ensuring that security practices are upheld and quality is preserved from development through to deployment and beyond.
- **Efficient Automation and Deployment:** Automated provisioning and deployment of smart contracts across various environments ensure swift and validated operations, improving the scalability and reliability of blockchain-based applications while maintaining rigorous security standards.

These key benefits highlight the transformative potential of integrating an adaptive, cognitive vulnerability detection approach into DevSecOps for blockchain systems, ensuring more secure, efficient, and reliable smart contract development and deployment.

III. SMART CONTRACT COMMON ISSUES

As the adoption of smart contracts continues to grow, they have become a prime target for malicious actors seeking to exploit security vulnerabilities. According to Dune Analytics [29], by March 2020, over two million smart contracts had been deployed on the Ethereum platform alone, highlighting the rapid expansion and widespread use of this technology. However, this surge in deployment has also led to an increase in security breaches, with vulnerabilities in smart contracts resulting in multi-billion-dollar financial losses across the blockchain ecosystem.

To better understand and mitigate these risks, smart contract vulnerabilities are typically classified into three hierarchical layers:

- **Code Layer:** Includes bugs and logic flaws introduced during development, such as reentrancy, integer overflows, and improper access control.
- **Execution Layer:** Refers to issues that arise during the execution of the smart contract, often due to unexpected interactions between functions, gas limit constraints, or misinterpretation of the Ethereum Virtual Machine (EVM) behavior.
- **Block Dependency Layer:** Involves vulnerabilities stemming from a contract's reliance on external blockchain factors such as timestamps, block numbers, or the order of transactions, which can be manipulated by miners or external agents.

This classification helps developers, researchers, and security analysts to adopt a structured approach in analyzing and addressing the complex threat landscape of smart contract ecosystems.

A. CODE LAYER

1) REENTRANCY

Similar to traditional programming environments, smart contracts often rely on cross-contract or cross-function invocations to implement complex business logic. These interactions, however, introduce significant security risks, particularly when they involve sensitive operations such as fund transfers or token deposits. Due to Ethereum's default behavior, such operations automatically trigger the fallback or receive function of the recipient contract. This opens the door for malicious actors to exploit the execution flow during an external call.

In a reentrancy attack, an attacker can exploit this mechanism by injecting malicious code into the fallback function of a contract. When a vulnerable contract executes an external call, such as a token transfer, the attacker's fallback function recursively calls the original contract before the first execution is completed. This allows the attacker to drain funds repeatedly before the contract updates its internal state. A notorious example is the DAO Attack [30], where an attacker exploited a reentrancy vulnerability in the DAO smart contract to steal approximately \$60 million worth of Ether. This incident not only led to massive financial losses but also forced the Ethereum community to implement a hard fork, resulting in the split between Ethereum and Ethereum Classic.

```
pragma solidity ^0.8.0;

contract ReentrancyVulnerable {
    mapping(address => uint256) public balances;

    // Deposit function to store Ether for a user
    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    // Withdraw function that is vulnerable to reentrancy
    function withdraw(uint256 _amount) public {
        require(balances[msg.sender] >= _amount,
            "Insufficient balance");

        // Call external contract (or self call)
        // before updating state
        payable(msg.sender).transfer(_amount);

        // Update the user's balance after the transfer
        balances[msg.sender] -= _amount;
    }
}
```

This attack highlights the critical importance of carefully handling external calls and implementing secure coding patterns, such as the checks-effects-interactions pattern and using reentrancy guards, to prevent such exploits. As smart contracts continue to grow in complexity and adoption, ensuring secure execution of cross-contract interactions remains a fundamental aspect of smart contract security.

2) ARITHMETIC

Smart contracts are particularly vulnerable to integer-related errors, including multiplication overflow, addition overflow, and subtraction underflow. In most smart contract platforms, such as Ethereum, integers are of fixed size and constrained by the boundaries of their data types (e.g., uint8, uint256).

When arithmetic operations exceed these limits, they cause an overflow or underflow, where the values wrap around to unintended results. For instance, an addition that exceeds the maximum value of a 256-bit unsigned integer would reset the value to zero or a small number, leading to unexpected and potentially exploitable outcomes.

These types of vulnerabilities are particularly dangerous in financial operations, where precise calculations are critical. A well-documented example of this threat occurred in the BeautyChain (BEC) token incident, where attackers exploited an integer overflow vulnerability to mint an unlimited number of tokens. This manipulation rendered the token worthless and resulted in a total collapse of its market value [5].

Such vulnerabilities stem from the fact that, in earlier versions of Solidity (prior to 0.8.0), arithmetic operations did not include built-in overflow or underflow checks. As a mitigation strategy, developers are encouraged to manually audit smart contract code and use reliable arithmetic libraries, such as SafeMath [31]. SafeMath provides wrapper functions that automatically revert the transaction in case of arithmetic errors, ensuring the integrity of calculations.

```
pragma solidity ^0.8.0;

contract ArithmeticVulnerable {
    uint public balance;

    // Deposit function to add funds to
    // the contract
    function deposit(uint _amount) public {
        // Adding amount to the balance
        balance += _amount;
    }

    // Withdraw function that is vulnerable to
    // underflow
    function withdraw(uint _amount) public {
        require(balance >= _amount,
            "Insufficient balance");
        // Subtracting amount from the balance
        balance -= _amount;
    }

    // Function to transfer funds to
    // another contract, vulnerable to overflow
    function transfer(uint _amount) public {
        // Check to prevent overflow
        require(balance + _amount >= balance,
            "Overflow error");
        balance += _amount;
    }
}
```

Moreover, with newer versions of Solidity, built-in overflow protections are enabled by default, but for legacy contracts or platforms without this feature, SafeMath and similar tools remain essential. In addition to library support, formal verification tools and automated static analyzers can also help detect arithmetic vulnerabilities early in the development cycle, thereby reducing the attack surface and enhancing the overall security of smart contracts.

3) ACCESS CONTROL

Smart contract functions and variables can be assigned various access restrictions to control visibility and interaction.

These restrictions are typically categorized as public, private, external, or internal, each defining the scope of access for different users and contracts. For example, public functions can be accessed by any external entity, while private functions can only be accessed within the same contract.

To enforce appropriate access control, modifiers like `onlyOwner` and `onlyAdmin` are commonly used. These modifiers restrict access to specific functions or variables based on the role or identity of the caller. The `onlyOwner` modifier, for instance, ensures that only the contract owner can call critical functions, such as transferring ownership or withdrawing funds, while `onlyAdmin` limits access to administrative functions, thereby enhancing security and preventing unauthorized actions.

The application of such modifiers is crucial for maintaining the integrity and confidentiality of smart contract operations. If a function lacks proper access control, it may become vulnerable to unauthorized interactions, which can lead to severe security issues. Malicious actors can exploit these vulnerabilities by interacting with exposed functions, potentially gaining control over assets or altering contract behavior. For example, an attacker might trigger a critical function designed for the contract owner, such as transferring tokens, leading to financial losses or contract compromise.

```
pragma solidity ^0.8.0;

contract AccessControlVulnerable {
    address public owner;

    constructor() {
        // The contract creator is the initial owner
        owner = msg.sender;
    }

    // Function to set a new owner,
    // but it lacks proper access control
    function setOwner(address _newOwner) public {
        // Any user can call this function and
        // set a new owner
        owner = _newOwner;
    }
}
```

Thus, ensuring that smart contract functions are adequately protected by access control mechanisms is a fundamental aspect of secure contract design. In addition to using modifiers, it is advisable to conduct thorough code audits and leverage automated tools to identify potential access control weaknesses, ensuring that only authorized parties can execute sensitive functions.

4) MISHANDLED EXCEPTION

Exception handling is a critical aspect of smart contract development, as it ensures that contracts can respond gracefully to errors and unforeseen events, thereby improving overall program reliability and robustness. There are three primary categories of exceptions that smart contracts may encounter:

- **Gas Exhaustion:** In Ethereum-based smart contracts, gas refers to the computational cost required to execute or deploy a contract. If the gas provided for a transaction

is insufficient to complete the execution, the transaction will fail, and an exception will be triggered. This ensures that contracts do not consume excessive resources, preventing indefinite execution or accidental resource depletion.

- **Call-Stack Overflow:** This exception occurs when the number of function calls exceeds the maximum allowed call stack depth. The call stack is the structure that stores the execution context of function calls. If a function recursively calls itself without a base case, or if a series of nested calls exceeds the system limit, a stack overflow will occur, halting further execution.
- **Exception Statements:** Smart contracts are equipped with specific error handling mechanisms that allow developers to capture and manage exceptions through predefined statements. These statements define how the contract should behave when encountering unexpected conditions, such as reverting to a previous state and providing an error identifier as feedback.

Generally, when an exception is encountered in a smart contract, the transaction is reverted to its previous state, undoing any changes made during the execution process. This rollback mechanism prevents inconsistent or corrupt data from being written to the blockchain. However, the smart contract that initiated the transaction may not always receive detailed information about the exception from the contract it called. In some cases, function calls may return a false value or an empty result instead of triggering an explicit exception, which could lead to insufficient feedback regarding the nature of the failure.

```
pragma solidity ^0.8.0;

contract MishandledException {
    mapping(address => uint256) public balances;

    // Deposit function to add Ether to
    // the contract balance
    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    // Withdraw function that mishandles
    // the exception if transfer fails
    function withdraw(uint256 _amount) public {
        require(balances[msg.sender] >= _amount,
            "Insufficient balance");

        // Transfer Ether but mishandle
        // the exception if it fails
        bool success = payable(msg.sender).send(_amount);
        if (!success) {
            revert("Withdrawal failed");
        }

        balances[msg.sender] -= _amount;
    }
}
```

Therefore, careful exception handling is vital when invoking external functions. Developers must ensure that they verify the returned values from function calls thoroughly and handle them consistently to detect errors. This includes checking whether the transaction was successful and, in cases of failure, taking the necessary actions to log errors, notify users, or initiate alternative execution paths. Moreover,

integrating revert mechanisms with clear error messages, such as `require()`, `assert()`, and `revert()` in Solidity, can significantly improve the contract's transparency and help with debugging and error recovery.

5) DENIAL OF SERVICE

A common vulnerability in smart contracts is Denial of Service (DoS), which causes the contract to become unresponsive and unable to provide its intended services for an extended period [32]. DoS attacks in smart contracts can be categorized into three primary types:

- **External Function Dependency:** When a smart contract depends on the outcome of an external function to update its state, it becomes vulnerable to a DoS attack if the contract fails to properly handle any failure of the external function. If the external call does not return the expected result or encounters an error, the contract may become stuck, unable to complete its operations.
- **Gas Limit Attacks:** A DoS attack can also occur when a contract-initiated transaction is blocked because the gas required to execute the transaction exceeds the predefined threshold. In such cases, a malicious attacker can manipulate the gas cost, artificially inflating it, which leads to the transaction failing due to reaching the gas limit. This obstructs the contract's ability to function properly and may prevent essential transactions from being completed.
- **Owner Account Vulnerability:** If a smart contract's owner account is not sufficiently protected, it becomes susceptible to attacks. A compromised or poorly secured owner account can be manipulated by malicious actors, posing a significant threat to the entire contract. In the worst case, this can result in the permanent freezing of tokens or assets controlled by the contract, causing substantial financial losses and disrupting the contract functionality.

```
pragma solidity ^0.8.0;

contract DenialOfServiceVulnerable {
    // List of users to distribute rewards to
    address[] public users;
    // Amount of reward to distribute
    uint256 public rewardAmount;

    // Function to add users to the list
    function addUser(address _user) public {
        users.push(_user);
    }

    // Function to set the reward amount
    function setRewardAmount(uint256 _amount) public {
        rewardAmount = _amount;
    }

    // Function to distribute rewards to all users (vulnerable to DoS)
    function distributeRewards() public {
        for (uint256 i = 0; i < users.length; i++) {
            payable(users[i]).transfer(rewardAmount);
        }
    }

    // Receive function to accept Ether for rewards
    receive() external payable {}
}
```

To mitigate the risks associated with DoS attacks, smart contract developers must implement proper error handling, robust access controls, and gas optimization strategies. Ensuring that external dependencies are secure and that

gas limits are accurately calculated can help reduce the likelihood of these attacks. Additionally, securing the owner account and incorporating multi-signature or time-locked mechanisms can strengthen the contract's resistance to potential manipulation.

6) TYPE MISMATCH

During program execution, smart contracts cannot directly raise exceptions for type mismatches. While smart contracts are typically checked for type consistency during the compilation process, where certain errors can be caught, these checks alone are not always sufficient to prevent all potential issues. As a result, even though the contract's source code may be syntactically correct, unexpected type-related vulnerabilities can still arise at runtime, especially when interacting with external contracts or when performing complex calculations that involve multiple data types.

```
pragma solidity ^0.8.0;

contract TypeMismatchExample {
    uint256 public number;

    // Function to set a number
    function setNumber(uint256 _number) public {
        number = _number; // Correct
    }

    // Function to add a value (with a type mismatch error)
    function addValue() public pure returns (uint256) {
        uint256 value = 10;
        uint8 smallValue = 5;

        // Type mismatch error:
        // cannot add uint256 and uint8 directly
        return value + smallValue;
    }
}
```

To address this issue, additional audits and testing are required to ensure that type mismatches do not lead to unforeseen vulnerabilities. This includes verifying that inputs to functions are of the correct types and ensuring that any conversions between different data types (e.g., integer to string, or address to uint) are handled safely and correctly. Moreover, developers should adopt practices such as unit testing, static analysis, and formal verification to systematically check for type-related issues that could otherwise be overlooked during compilation.

By integrating these additional security measures, developers can reduce the risk of introducing type mismatch vulnerabilities, thereby ensuring that the contract behaves as expected across all possible scenarios and interactions.

7) UNKNOWN FUNCTION CALL

When a caller contract makes a call to a callee contract using an incompatible function name or parameter values, the fallback function in the caller contract is automatically triggered. The fallback function is a special function in smart contracts that is executed when a contract does not recognize the called function or when no data is sent with the call. While this is a mechanism to handle unforeseen or unintended function calls, it can introduce security vulnerabilities.

A significant security concern arises if a malicious actor exploits the fallback function. By carefully designing and hiding malicious code within the fallback function, an attacker can bypass the intended flow of the contract and potentially manipulate the contract's state or steal funds. Since the fallback function is often triggered by an unintended interaction, this situation can lead to unexpected contract behavior, making the contract susceptible to attacks.

For example, an attacker could send a call to the contract with incorrect parameters, which would trigger the fallback function. If the fallback function contains harmful logic, the attacker could exploit this to drain funds, manipulate variables, or disrupt the contract's normal operation.

```
pragma solidity ^0.8.0;

contract UnknownFunctionExample {
    // Function to perform an action
    function performAction(uint256 value)
        public pure returns (string memory) {
        if (value > 0) {
            return "Action performed successfully!";
        }
        return "Invalid value!";
    }

    // Fallback function
    fallback() external payable {
        // Fallback logic
    }
}

pragma solidity ^0.8.0;

interface UnknownFunctionExampleInterface {
    function performAction(uint256 value)
        external returns (string memory);
}

contract ExternalCaller {
    function callUnknownFunction(address target, uint256 value)
        public returns (string memory) {
        // Misspelled function name
        UnknownFunctionExampleInterface(target).performAction(value);
    }
}
```

To mitigate this risk, smart contract developers should minimize the use of fallback functions or ensure that they are carefully designed with proper validation and limitation mechanisms. This includes:

- Restricting access to the fallback function by implementing strict access control.
- Implementing checks to ensure that only valid function names and parameters are accepted.
- Limiting the operations allowed within the fallback function to prevent unwanted or malicious actions.
- Auditing and thoroughly testing the fallback function to identify any potential security weaknesses.

8) TOKEN FROZEN

Token freezing refers to a situation where users are unable to transfer their tokens due to restrictions enforced by the smart contract. This can occur either intentionally, as a part of the contract's design (e.g., in cases where certain tokens are temporarily locked or frozen for specific purposes like governance or compliance), or accidentally due to a bug in the contract's logic.

Unintended freezing of tokens is a significant security concern because it can render users' assets inaccessible or

frozen for an indefinite period. This happens when certain conditions or functions that control the transferability of tokens are improperly implemented or not properly handled, either due to a flaw in the contract's logic or unexpected behavior caused by interactions with external contracts or users.

For example, a token contract may mistakenly freeze user balances due to errors in the access control logic or transaction conditions. If the contract fails to validate conditions like ownership or authorization correctly, users may find themselves unable to transfer tokens even when they have the right to do so.

```
pragma solidity ^0.8.0;

contract TokenFrozenExample {
    string public name = "FrozenToken";
    string public symbol = "FTK";
    uint256 public totalSupply = 1000000;
    mapping(address => uint256) public balances;
    mapping(address => bool) public frozenAccounts;

    event Transfer(address indexed from,
        address indexed to, uint256 value);
    event AccountFrozen(address indexed account, bool isFrozen);

    constructor() {
        // Assign all tokens to the contract deployer
        balances[msg.sender] = totalSupply;
    }

    // Function to transfer tokens
    function transfer(address to, uint256 value)
        public returns (bool) {
        require(!frozenAccounts[msg.sender],
            "Sender account is frozen");
        require(!frozenAccounts[to],
            "Recipient account is frozen");
        require(balances[msg.sender] >= value,
            "Insufficient balance");

        balances[msg.sender] -= value;
        balances[to] += value;

        emit Transfer(msg.sender, to, value);
        return true;
    }

    // Function to freeze an account
    // (can only be called by the owner)
    function freezeAccount(address account, bool isFrozen) public {
        frozenAccounts[account] = isFrozen;
        emit AccountFrozen(account, isFrozen);
    }
}
```

To mitigate the risks associated with token freezing, developers can implement the following best practices:

- **Thorough Testing:** Ensure that the smart contract undergoes comprehensive testing, including edge cases, to check if the freezing mechanism functions as expected.
- **Clear Logic for Freezing Mechanism:** Design the token freezing mechanism clearly and explicitly in the contract's logic, so that freezing is applied only when necessary and can be easily managed.
- **Fail-safes:** Implement fallback mechanisms to unfreeze tokens in case of erroneous or malicious locking, allowing users to regain control of their assets.
- **Audit and Review:** Regularly audit the contract to ensure that the freezing mechanism is not vulnerable to bugs or exploits, especially if the logic involves complex conditions or external calls.

9) REPLAY ATTACK

Transactions initiated on one blockchain may still be considered valid on another blockchain, provided that key elements, such as the address, private key, algorithm, and transaction format, remain unchanged before a hard fork. This creates a potential vulnerability, where users can exploit this condition to double-spend tokens or replicate transactions across multiple blockchains.

For example, if a user receives a specific amount of tokens from another user on one blockchain, they may then switch to a different blockchain after a hard fork, where the same transaction remains valid. This results in the fraudulent transfer of an identical amount of tokens to the user's account on the second blockchain, effectively duplicating the transaction. This scenario is a form of double-spending, where the same assets are spent multiple times, causing significant financial harm to the blockchain ecosystem.

```
pragma solidity ^0.8.0;
contract RelayAttackExample {
    mapping(address => uint256) public balances;

    event Deposited(address indexed user, uint256 amount);
    event Executed(address indexed user,
        address indexed recipient, uint256 amount);

    // Deposit function
    function deposit() public payable {
        balances[msg.sender] += msg.value;
        emit Deposited(msg.sender, msg.value);
    }

    // Function to execute a transfer with a signed message
    function executeTransfer(
        address recipient,
        uint256 amount,
        uint8 v,
        bytes32 r,
        bytes32 s
    ) public {
        // Reconstruct the message hash
        bytes32 messageHash = keccak256(
            abi.encodePacked(msg.sender, recipient, amount)
        );

        // Recover the signer from the signature
        address signer = ecrecover(messageHash, v, r, s);

        // Verify the signer and transfer the funds
        require(signer == msg.sender, "Invalid signature");
        require(balances[signer] >= amount, "Insufficient balance");

        balances[signer] -= amount;
        payable(recipient).transfer(amount);

        emit Executed(signer, recipient, amount);
    }
}
```

This issue is particularly concerning for cross-chain transactions or in the event of hard forks where the same tokens or assets may exist on two different versions of a blockchain. To prevent such fraudulent activity, blockchain systems can implement several safeguards, including:

- **Cross-chain Validation:** Implementing mechanisms that ensure transactions are validated across multiple chains, preventing the same transaction from being executed on more than one blockchain.
- **Transaction Replay Protection:** Using techniques such as replay protection to prevent transactions from being accepted on a different blockchain after a hard fork or split.

- **Unique Identifiers:** Ensuring that transaction identifiers are unique to the specific chain, preventing the same transaction from being replicated on another chain.
- **Monitoring and Detection:** Establishing monitoring systems that can detect and flag suspicious or replicated transactions, providing an additional layer of security against such attacks.

B. EXECUTION LAYER

1) SHORT ADDRESS

In scenarios where a smart contract address is shorter than a regular address, the execution machine may automatically pad the address with additional digits from the subsequent parameter to fill the required length. This process involves extracting the necessary coding digits from the following parameter to complete the address, after which the entire string of binary codes is reset to zero. This allows the next parameter to shift left by a few bytes to maintain alignment.

However, this behavior introduces a potential vulnerability during transfer operations. If a transfer occurs while this padding process is underway, there is a risk that the smart contract could inadvertently be transferred to an attacker. The amount transferred may exceed the originally intended value due to the misalignment of parameters, potentially allowing the attacker to gain control of the contract or exploit the excess funds.

This issue arises from the way the execution machine handles parameter shifts and padding during smart contract operations. It creates a scenario where the expected transaction data may not align with the correct values, leading to unintended transfers or other exploits.

```
pragma solidity ^0.8.0;

contract ShortAddressExample {
    mapping(address => uint256) public balances;

    event Transfer(address indexed from,
        address indexed to, uint256 amount);

    // Function to transfer tokens
    function transfer(address to, uint256 amount) public {
        require(balances[msg.sender] >= amount, "
            Insufficient balance");

        balances[msg.sender] -= amount;
        balances[to] += amount;

        emit Transfer(msg.sender, to, amount);
    }
}
```

To mitigate such risks, smart contract developers should carefully validate and test the alignment of contract parameters, ensuring that padding or address shortening does not introduce vulnerabilities during transfer operations. Additionally, applying strict checks for address formats and transaction parameters can help prevent these kinds of misalignments and protect the integrity of smart contract transactions.

2) TOKEN LOSS

If a smart contract transfers tokens to a unique and unrelated address that is not associated with any other

users or contracts, there is a high likelihood that the tokens will be irretrievably lost. This situation occurs because the transferred tokens are effectively sent to an address that lacks a private key or control, meaning there is no entity able to access, manage, or transfer those tokens again.

In blockchain systems, transactions are typically irreversible once they are confirmed, and if the destination address is not under the control of any active user or smart contract, the tokens become inaccessible. Since blockchain addresses are not inherently linked to any centralized authority or recovery mechanism, tokens sent to an invalid or unclaimed address are essentially “burned” and cannot be recovered.

```
pragma solidity ^0.8.0;

interface IERC20 {
    function transferFrom(address sender, address recipient,
        uint256 amount) external returns (bool);
    function transfer(address recipient, uint256 amount)
        external returns (bool);
}

contract TokenLossExample {
    IERC20 public token;
    mapping(address => uint256) public balances;

    constructor(address tokenAddress) {
        token = IERC20(tokenAddress);
    }

    // Deposit tokens into the contract
    function deposit(uint256 amount) public {
        require(amount > 0,
            "Amount must be greater than zero");

        // Transfer tokens from the user to the contract
        token.transferFrom(msg.sender, address(this), amount);

        // Update the user's balance
        balances[msg.sender] += amount;
    }

    // Withdraw tokens from the contract
    function withdraw(uint256 amount) public {
        require(amount > 0,
            "Amount must be greater than zero");
        require(balances[msg.sender] >= amount,
            "Insufficient balance");

        // Transfer tokens back to the user
        // BUG: Tokens are sent to address(0)
        // instead of msg.sender
        token.transfer(address(0), amount);

        // Update the user's balance
        balances[msg.sender] -= amount;
    }
}
```

To prevent such losses, smart contract developers should implement measures to ensure that tokens are only sent to valid and correctly configured addresses. This includes:

- **Safe Transfer Mechanisms:** Using built-in or third-party libraries that handle token transfers more safely, ensuring that no tokens are sent to unassociated or unintended addresses.
- **Transaction Reversibility Checks:** Where possible, incorporating mechanisms that allow the contract to verify or cancel transfers before they are finalized, especially in case of errors.
- **Address Validation:** Ensuring that the recipient address is valid and associated with a user or contract before executing the transfer.
- **Recipient Confirmation:** Adding extra layers of confirmation within the contract to verify that the transfer is intentional and the address is correct.

By addressing these issues, smart contracts can minimize the risk of transferring tokens to unintended addresses, helping to preserve the integrity and security of token transactions.

3) CALL-STACK OVERFLOW

When a smart contract triggers an external function or performs a self-call, it adds to the call stack, which has a limited capacity. The call stack is a structure that keeps track of function calls and their execution states. Each function call occupies a portion of the stack, and once this stack exceeds its limit, a “call-stack overflow” occurs, causing the contract to fail.

If an attacker orchestrates a series of nested function calls within a smart contract, they can deliberately increase the depth of the call stack, potentially exceeding its upper threshold. This tactic, known as a “call-stack attack,” can cause the contract to run out of space on the call stack, leading to a failure in execution. In some cases, this failure can result in unintended consequences, such as preventing the contract from completing critical functions or allowing the attacker to manipulate the contract’s behavior.

These vulnerabilities are particularly concerning in cases where the contract performs self-calls or calls external contracts without adequately limiting the depth of the call stack. The stack overflow can lead to unexpected behaviors, including the disruption of contract functionality or exploitation of the system’s logic.

```
pragma solidity ^0.8.0;

contract CallStackOverflow {
    address public owner;

    constructor() {
        owner = msg.sender;
    }

    // Recursive function call
    function callRecursively(address target,
        uint256 depth) public {
        require(msg.sender == owner,
            "Only owner can call this function");

        if (depth > 0) {
            // Recursive call
            CallStackOverflow(target).callRecursively(target,
                depth - 1);
        }
    }
}
```

To mitigate such risks, developers should:

- **Limit Recursion:** Avoid unbounded recursion or excessive nesting of function calls, which can easily cause the stack to overflow.
- **Check Call Depth:** Implement checks to limit the number of nested calls or recursion before triggering external calls or self-calls.
- **Gas Limit Management:** Ensure that transactions and function calls have appropriate gas limits to avoid triggering excessive function calls that could exhaust the stack.

- **Use Reentrancy Guards:** Employ reentrancy guards to prevent attackers from exploiting the contract through recursive calls during sensitive operations.

4) TRANSACTION ORIGIN USE

In smart contracts, global variables such as `tx.origin` can be used to trace the origin of a transaction. `tx.origin` returns the address of the account that initiated the transaction, providing a way for contracts to identify the external account involved in the transaction. However, relying on `tx.origin` for authentication and authorization introduces significant security risks.

One of the key issues is that `tx.origin` is not limited to the direct caller of the contract but can be traced back to the account that initiated the entire transaction chain. This opens the door for malicious actors to exploit the behavior of a contract if it incorrectly assumes that `tx.origin` is the only trustworthy indicator of the originator.

For example, consider a situation where a contract checks `tx.origin` to authorize the transfer of tokens. An attacker can exploit this by crafting a transaction where their malicious contract calls the vulnerable contract, and through nested calls, triggers the transfer of tokens. Since `tx.origin` refers to the account that initially started the transaction (even if that account is not directly interacting with the vulnerable contract), the attacker can trick the contract into allowing unauthorized actions under the guise of the legitimate user’s account.

```
pragma solidity ^0.8.0;

contract TxOriginExample {
    address public owner;

    constructor() {
        // Set the deployer as the owner
        owner = tx.origin;
    }

    // Function to withdraw funds
    function withdraw() public {
        require(tx.origin == owner,
            "Only the owner can withdraw funds");
        payable(owner).transfer(address(this).balance);
    }

    // Allow the contract to receive Ether
    receive() external payable {}
}
```

To avoid the exploitation of `tx.origin`, developers should use `msg.sender` instead of `tx.origin` for authentication and authorization. `msg.sender` refers to the immediate caller of the function and is more secure, as it reflects the direct interaction with the contract, making it harder for attackers to impersonate legitimate users through nested calls.

Moreover, developers can implement other security mechanisms, such as:

- **Access control modifiers** (e.g., `onlyOwner`, `onlyAdmin`) to ensure only authorized accounts can interact with critical functions.

- Reentrancy guards to prevent nested calls and potential exploits related to reentrancy attacks.
- Secure auditing to identify potential vulnerabilities that could be exploited using global variables like tx.origin.

By avoiding the use of tx.origin for sensitive operations and ensuring the proper implementation of access control, smart contract developers can significantly reduce the risk of such attacks.

C. BLOCK DEPENDENCY LAYER

1) TIMESTAMP DEPENDENCY

Smart contracts frequently rely on the block timestamp provided by the blockchain network to enforce time-sensitive logic. These timestamps, embedded into each block by the miner or validator and verified by other network participants, offer a unified way to synchronize contract behavior with time-based conditions. Since all transactions in a given block share the same timestamp, this approach ensures consistent execution across smart contracts, particularly for features such as deadlines, auction cutoffs, vesting schedules, and time locks.

However, depending on block timestamps for critical time-based logic introduces a notable vulnerability. The reason lies in the flexibility granted to miners or validators during block production, they can choose the timestamp for the block they validate, as long as it falls within protocol-defined boundaries (typically a few seconds ahead or behind the actual time). While these margins are small, they are often sufficient for malicious actors to manipulate the system in ways that serve their interests.

This opens the door to several potential exploits in contracts that enforce time-based conditions:

- **Auction Time Manipulation:** In smart contracts that implement auctions, the winner is typically the highest bidder within a defined time window. A malicious miner could extend this window by setting the timestamp slightly forward, allowing themselves or an accomplice to submit a winning bid after other participants have stopped bidding, effectively altering the outcome unfairly.
- **Premature Token Vesting:** Contracts that manage token vesting schedules based on time may be exploited if a validator manipulates the timestamp to make it appear as if more time has passed. This can result in premature unlocking of vested tokens, undermining lock-up periods designed to ensure fair distribution and gradual release.
- **Bypassing Time Locks:** Smart contracts often delay critical actions using time locks that prevent certain functions from executing until a specific future timestamp. If a miner sets the timestamp to just before the lock expires, they can bypass the delay, potentially gaining early access to restricted operations or information.

```
pragma solidity ^0.8.0;
contract TimestampDependencyExample {
    address public owner;
    address[] public players;

    constructor() {
        owner = msg.sender;
    }

    // Enter the lottery
    function enter() public payable {
        require(msg.value == 1 ether,
            "Must send 1 ether to enter");
        players.push(msg.sender);
    }

    // Pick a winner based on block.timestamp
    function pickWinner() public {
        require(msg.sender == owner,
            "Only the owner can pick the winner");
        require(players.length > 0,
            "No players in the lottery");

        uint256 index = uint256(keccak256(
            abi.encodePacked(block.timestamp,
                block.difficulty,
                players))) % players.length;
        address winner = players[index];

        payable(winner).transfer(address(this).balance);

        // Reset the lottery
        players = new address[](0);
    }
}
```

To protect smart contracts from timestamp manipulation attacks, developers can implement several mitigation techniques:

- **Use of Time Oracles:** Instead of solely relying on block timestamps, developers can incorporate time oracles, trusted external sources that provide accurate and verifiable time data. These oracles may pull time from real-world clocks or decentralized time protocols, offering greater reliability and resistance to manipulation.
- **Stricter Timestamp Validation:** Contracts can introduce logic that checks whether a given block timestamp falls within an acceptable range relative to a known or expected value. This includes comparing the timestamp to previous blocks or enforcing tolerances that limit how far it can deviate from the current system time.
- **Delayed Execution Mechanisms:** Implementing a delay between a triggered action and its execution can reduce the impact of timestamp manipulation. For instance, a contract might queue a request and only allow its execution after several blocks, mitigating the influence of any single manipulated timestamp.
- **Decentralized Time Aggregation:** Platforms can leverage decentralized oracles that combine time data from multiple independent sources. This aggregation reduces the likelihood that any single actor, such as a miner or validator, can control the time input used by the contract, enhancing trust in time-sensitive operations.
- **Redundant and Cross-Verified Time Sources:** To further strengthen security, contracts can cross-check time data from multiple sources. For example, a smart contract could require that the block timestamp be corroborated

by several subsequent blocks or confirmed by an external oracle before allowing certain time-sensitive actions.

By acknowledging and addressing the limitations of block timestamps, developers can reduce the attack surface for time-based vulnerabilities. Implementing layered and redundant time validation strategies ensures that smart contracts behave reliably and securely, even in adversarial environments where certain participants may attempt to manipulate execution conditions for their own gain.

2) BLOCK DEPENDENCY

Instead of relying on pre-existing functions for random number generation, smart contracts sometimes use block parameters such as the block number, block timestamp, or block hash as inputs to create a random number. While this method is commonly used due to its simplicity, it introduces significant vulnerabilities. Since block parameters are deterministic and can be influenced or predicted by miners or validators, attackers can manipulate these values before the block is finalized. This results in the generation of predictable random numbers.

```
pragma solidity ^0.8.0;

contract BlockDependencyFixed {
    address public owner;
    mapping(address => uint256) public lastClaimedTime;

    constructor() {
        owner = msg.sender;
    }

    // Reward claim function based on time
    function claimReward() public {
        require(block.timestamp > lastClaimedTime[msg.sender] + 1-days,
            "Wait for 1-day between claims");

        // Send reward
        payable(msg.sender).transfer(1 ether);

        // Update the last claimed time
        lastClaimedTime[msg.sender] = block.timestamp;
    }

    // Allow the contract to receive Ether
    receive() external payable {}
}
```

By exploiting this vulnerability, malicious actors can anticipate the outcome of random number generation, allowing them to influence the results in their favor. This could lead to unfair advantages in systems like gambling contracts, lotteries, or auctions, where randomness is critical to determining outcomes. For example, in a lottery contract, an attacker might predict the winning number, or in an auction, they could manipulate the random selection of winners.

To mitigate this issue, it's recommended that smart contracts rely on external randomness sources like oracle services that provide verifiable and tamper-proof randomness. These solutions ensure that the generated random numbers are unpredictable and cannot be manipulated by any single entity.

3) TRANSACTION ORDER DEPENDENCY

The state of a smart contract is highly dependent on the sequence in which transactions are executed. If the order of these transactions is altered or incorrect, it can result in unintended and harmful consequences for the

contract's functionality. The final state of the smart contract is determined by the execution order of transactions, which is confirmed by the network nodes in the blockchain. However, this creates a vulnerability that can be exploited by malicious attackers.

By monitoring transactions within a block, attackers can manipulate the execution order, potentially altering the state of the contract in a way that benefits them. This could lead to a variety of malicious outcomes, such as front-running, where an attacker executes a transaction with the intent of influencing the outcome of a previously submitted transaction, or reordering transactions, which might exploit the contract's logic to gain an unfair advantage.

```
pragma solidity ^0.8.0;

contract Auction {
    address public owner;
    uint public highestBid;
    address public highestBidder;
    uint public auctionEndTime;

    constructor(uint _auctionDuration) {
        owner = msg.sender;
        auctionEndTime = block.timestamp + _auctionDuration;
    }

    // Function to place a bid
    function placeBid() public payable {
        require(block.timestamp < auctionEndTime,
            "Auction has ended");
        require(msg.value > highestBid,
            "Bid must be higher than the current highest bid");

        // Refund previous highest bidder
        payable(highestBidder).transfer(highestBid);

        // Update the highest bid and bidder
        highestBid = msg.value;
        highestBidder = msg.sender;
    }

    // End the auction and transfer the item
    function endAuction() public {
        require(msg.sender == owner,
            "Only the owner can end the auction");
        require(block.timestamp >= auctionEndTime,
            "Auction has not ended yet");

        // Transfer the item to the highest bidder
        // (simulated by sending funds)
        payable(owner).transfer(highestBid);
    }
}
```

For example, in a decentralized exchange (DEX), an attacker might manipulate the order of transactions to front-run a trade and profit from price differences, or in a contract involving auctions, an attacker could reorder bids to secure the highest bid at a lower cost. This vulnerability introduces significant security risks, as attackers can undermine the integrity of the smart contract, leading to financial losses or disruption of services.

To mitigate this risk, smart contracts should implement strategies to ensure transaction ordering integrity, such as leveraging timestamp-based ordering, commitment schemes, or external ordering services to prevent manipulation of the transaction sequence.

IV. VULNERABILITY DETECTION TECHNIQUES

To protect smart contracts from exploitation by malicious actors, researchers have increasingly focused on comprehensive analysis techniques applied to both source code and compiled bytecode. This proactive strategy is crucial for

identifying and addressing vulnerabilities during the development phase, significantly reducing the risk of exploitation once smart contracts are deployed to production. Over the years, a diverse range of detection techniques has emerged, encompassing formal verification, fuzz testing, symbolic execution, intermediate representation (IR) analysis, and deep learning-based methods. These tools and methodologies each offer unique advantages in pinpointing logical errors, security flaws, and unsafe behaviors within contract logic.

Beyond these traditional methods, new and rapidly evolving technologies are being integrated into smart contract security. Quantum computing, for instance, presents the possibility of drastically improving vulnerability detection by parallelizing symbolic execution processes and solving complex satisfiability and verification problems more efficiently. In parallel, AI-driven techniques have shown remarkable success in automating tasks such as anomaly detection, vulnerability classification, and semantic analysis of smart contract logic, especially within large and dynamic codebases.

Pushing the frontier further, agentic AI introduces an autonomous, adaptive paradigm that surpasses the limitations of static analysis tools. These intelligent agents are capable of continuously monitoring deployed contracts, dynamically simulating contract behavior under varied conditions, and adjusting their analytical strategies in real time. They combine multiple techniques, such as symbolic reasoning, automated fuzzing, and behavioral learning, to discover previously unseen edge cases and potential vulnerabilities. Moreover, agentic agents can go beyond detection: they are capable of proposing or even implementing secure code patches, learning iteratively from past exploits, and refining their actions to anticipate evolving attack vectors. This continuous learning cycle positions agentic AI as a promising tool for real-time, lifecycle-spanning protection.

To provide a structured overview, Table 2 compares these approaches, traditional static and dynamic analysis techniques, quantum-augmented strategies, AI-based models, and agentic AI, by outlining their core principles, benefits, limitations, and use case scenarios in the context of smart contract security. This comparative analysis serves as both a reference and a guide for practitioners and researchers seeking to integrate robust security measures into smart contract development pipelines.

By combining classical methodologies with emerging computational paradigms, including AI and quantum capabilities, the security community can build resilient smart contract infrastructures that are proactive, adaptive, and responsive to the rapidly evolving threat landscape.

A. FORMAL VERIFICATION

Formal verification is widely regarded as one of the most precise methods for validating the authenticity and correctness of smart contracts. This approach utilizes formal logic languages to explicitly represent smart contracts and relies on

rigorous mathematical analysis to establish the soundness and reliability of their functionalities. Two prominent techniques within formal verification are model checking and deductive verification:

- Model checking involves systematically exploring the entire state space of a smart contract to identify all possible states. Each state is then analyzed individually to ensure that the contract exhibits the desired behavior or properties under all circumstances. This method provides a thorough examination of a contract's behavior across all potential execution paths, ensuring that no edge cases are overlooked.
- Deductive verification, in contrast, uses logical formulas and theorem proofs to define the desired properties of the contract. The technique employs evidence rules to rigorously validate these properties, ensuring that the contract adheres to its expected functionality and security requirements. By using logical reasoning, this method offers a formal guarantee that the contract will behave as intended under specific conditions.

Overall, formal verification relies on mathematical logic to verify that a code implementation meets predefined critical prerequisites. This process enhances the dependability and security of smart contracts by identifying potential vulnerabilities before they can be exploited.

Examples of tools that utilize formal verification techniques include ZEUS [33], which uses abstract interpretation, symbolic model checking, and constraint statements to efficiently validate the security of smart contracts. KEVM [15], another formal analysis framework, leverages the K framework to construct an executable formal specification. KEVM serves as a platform for developing semantic extensions and various analysis tools for smart contracts, further advancing the state of formal verification in blockchain security.

By incorporating formal verification into smart contract development, developers can ensure a higher level of standardization, rigor, and security, providing robust assurance that the contract will function securely and as intended in production environments.

B. SYMBOLIC EXECUTION

Symbolic execution is a powerful technique used to simplify the analysis of programs, particularly for detecting vulnerabilities in smart contracts. It replaces regular inputs within a program with symbolic inputs, allowing for a symbolic execution of every possible execution path. This process identifies which inputs trigger specific components of the program. As the program executes, an interpreter utilizes symbolic input values to derive constraints or expressions for the potential outcomes of each conditional branch, based on these symbols. By solving these constraints, it becomes possible to determine the set of symbolic inputs that can lead to particular execution paths or branches.

TABLE 2. Vulnerability detection tools.

Tool/ Platform	Techniques	Code layer								Execution layer				Block dependency layer			
		Reen- trancy	Arith- metic	Access con- trol	Excep- tion han- dling	Denial of Ser- vice	Type mis- match	Un- known func- tion call	Token frozen	Replay attack	Short ad- dress	Token loss	Call- stack over- flow	Transac- tion origin	Time stamp depen- dency	Block de- pen- dency	Transaction order de- pendency
ZEUS [33]	Static testing Formal verification Deep learning/AI	x	x							x					x	x	x
KEVM [15]	Static testing Formal verification Taint analysis																
Oyente [8]	Static testing Symbolic execution	x	x		x					x			x		x		x
Maian [9]	Static testing Dynamic testing Symbolic execution	x										x	x				
Osiris [10]	Static testing Symbolic execution	x	x		x					x			x		x		x
Securify [11]	Static testing Symbolic execution	x			x		x	x	x	x	x	x	x	x	x	x	x
Slither [12]	Static testing Intermediate representation Symbolic execution	x					x	x	x				x	x			
Contract Fuzzer [13]	Dynamic testing Fuzzing testing	x			x										x	x	
SmartCheck [17]	Static testing Intermediate representation	x	x	x		x		x	x					x	x		
SaferSC [34]	Static testing Deep learning/AI	x											x				
ReCheck [35]	Static testing Deep learning/AI	x															
Mythril [36]	Static testing Dynamic testing Symbolic execution	x	x		x	x				x		x		x	x	x	
ReGuard [37]	Dynamic testing Fuzzing testing	x															

When applied to smart contract vulnerability detection, symbolic execution follows a structured process divided into four main steps:

- Symbolize the contract's variable values: Replace actual values with symbolic representations to explore potential inputs.
- Interpret each instruction in the execution program: Detailed analysis of each step in the contract's execution process.
- Explore all executable paths: Traverse every possible execution path in the contract, updating the execution state and collecting path constraints along the way.
- Identify security concerns: Pinpoint vulnerabilities in the contract by analyzing the potential execution paths and path constraints.

One of the prominent tools that utilize symbolic execution is Oyente [8], a pioneering tool in the detection of smart contract vulnerabilities. Oyente uses symbolic execution to analyze the control flow graph (CFG) of a smart contract. By taking the smart contract's bytecode and state as input, Oyente emulates the Ethereum Virtual Machine (EVM), exploring various execution paths to detect vulnerabilities. This tool has been widely recognized for its ability to uncover vulnerabilities such as reentrancy, gas limit errors, and integer overflows.

Another noteworthy tool in this space is Maian [9], which takes a dynamic approach to vulnerability detection. Maian performs dynamic analysis by investigating a sequence of invocations during the contract's runtime. This method helps in identifying vulnerabilities that arise from the interaction between different parts of a contract during execution, especially those related to external calls and interactions between contracts.

Together, tools like Oyente and Maian play an important role in smart contract security, leveraging symbolic execution to uncover potential vulnerabilities before they can be exploited in a live environment.

C. INTERMEDIATE REPRESENTATION

Researchers are actively working to improve the accuracy of smart contract analysis by converting the source code or bytecode of contracts into an intermediate representation (IR), which encapsulates highly semantic information about the contract's behavior. This transformation allows for a more in-depth examination, making it easier to identify potential security issues that may not be apparent in the raw source code or bytecode.

Slither [12] is one such tool that uses this approach. It is a static analysis framework tailored for Ethereum smart contracts. Slither converts the contract's source code into an intermediate representation, which enables it to extract critical insights about the contract's logic. By analyzing this representation, Slither can efficiently identify a variety of vulnerabilities, such as reentrancy attacks, uninitialized

storage, and others that may pose risks to the contract's security.

Similarly, SmartCheck [17] is a flexible static analysis tool designed for smart contract security. It transforms the source code into an XML-based intermediate representation and uses XPath pattern analysis to detect vulnerabilities within this transformed version of the code. This method allows SmartCheck to identify common flaws such as improper access control and insufficient error handling, offering another layer of defense against potential exploits.

Both Slither and SmartCheck highlight the importance of intermediate representations in modern smart contract analysis. By focusing on these IRs, researchers and security experts can gain a deeper understanding of a contract's structure and behavior, significantly enhancing the detection of vulnerabilities and improving the overall security of smart contracts.

D. FUZZING TESTING

Fuzz testing is a widely adopted technique for analyzing software and identifying vulnerabilities. It works by generating a large number of test inputs and observing any abnormal behavior or crashes during program execution. One of the main advantages of fuzz testing is its simplicity and scalability, making it applicable across various types of software systems, including smart contracts.

For example, ContractFuzzer [13] is a dynamic analysis tool specifically designed for Ethereum smart contracts. It employs fuzz testing to identify potential security vulnerabilities in smart contracts. ContractFuzzer works by creating fuzzing inputs based on the Application Binary Interface (ABI) specification of the smart contract and developing a comprehensive test plan that systematically tests the contract for vulnerabilities. By sending these randomized inputs to the contract, it can uncover flaws like reentrancy issues, gas limit attacks, and other security risks, helping to ensure the robustness of smart contract implementations.

Fuzz testing, as demonstrated by ContractFuzzer [13], offers an efficient way to identify vulnerabilities early in the development process and plays a critical role in improving the security of smart contracts before deployment.

E. DEEP LEARNING

In recent years, there has been a noticeable shift toward leveraging deep learning techniques for program security analysis, and this approach has shown promising results. The advancements in deep learning have led to the development of a variety of security detection methods that utilize deep learning models, offering scalability and flexibility in addressing emerging vulnerabilities. These methods are particularly well-suited for detecting complex vulnerabilities that may be difficult to identify through traditional approaches.

One groundbreaking deep learning-based model is SaferSC [34], which focuses on identifying vulnerabilities in smart contracts. SaferSC is a pioneering model in this field, specifically designed to detect the types of vulnerabilities

first identified by Maian [9]. It examines the operation code (opcode) of smart contracts and uses a Long Short-Term Memory (LSTM) network to create a sequence model at the opcode level. This model allows SaferSC to accurately identify vulnerabilities by analyzing the sequential patterns in the contract's opcodes. This approach represents a significant leap forward in the integration of deep learning into smart contract security analysis.

Another deep learning-driven approach is ReCheck [35], which specializes in detecting reentrancy vulnerabilities in smart contracts. RecChecker transforms the contract source code into a contract snippet format, enabling it to capture essential semantic details and control flow dependencies within the contract. It then applies a Bidirectional Long Short-Term Memory (BLSTM) model to automatically identify reentrancy vulnerabilities by analyzing the flow of function calls and data within the contract. This method enhances the ability to detect complex vulnerabilities that may be missed by traditional static analysis tools.

These deep learning-based methods represent a cutting-edge approach to smart contract security, offering more accurate, scalable, and automated vulnerability detection capabilities compared to earlier techniques. As deep learning continues to evolve, its application in the realm of smart contract security is expected to expand, providing even more robust solutions for mitigating potential threats.

F. QUANTUM COMPUTING

Quantum computing holds the potential to revolutionize the field of smart contract vulnerability detection, surpassing classical methods in certain areas. One of the key advancements in this field is the use of Quantum Machine Learning (QML) techniques [38], such as quantum support vector machines and quantum neural networks, which excel in detecting complex and subtle vulnerability patterns that are often challenging for classical models to identify. These quantum models are capable of analyzing vast amounts of data more efficiently and can uncover intricate vulnerabilities that may otherwise go unnoticed.

In addition to pattern recognition, quantum computing enhances essential processes such as symbolic execution. By enabling the parallel evaluation of multiple execution paths, quantum computing significantly increases the efficiency of symbolic execution. This improvement accelerates the analysis of smart contracts, making it possible to explore a wider range of potential vulnerabilities in a shorter time frame.

Formal verification also benefits from quantum computing, as quantum algorithms are better equipped to solve complex satisfiability problems than their classical counterparts. These problems are central to verifying the correctness of smart contract behavior, and quantum techniques can provide a more robust and effective solution.

Furthermore, quantum computing has the potential to revolutionize cryptographic analysis. Many smart contracts

rely on cryptographic algorithms to secure transactions and ensure data integrity. Quantum computers can uncover weaknesses in these cryptographic algorithms, potentially exposing vulnerabilities that could be exploited by attackers. Additionally, quantum technologies play a crucial role in evaluating post-quantum cryptographic schemes, which are designed to resist attacks from future quantum-based systems. As quantum computing evolves, it will play an essential role in ensuring the resilience of smart contracts against the emerging threats posed by quantum technologies.

In summary, quantum computing presents significant opportunities for enhancing the security of smart contracts, enabling more efficient detection of vulnerabilities, improving formal verification processes, and strengthening cryptographic defenses. As quantum technologies advance, their integration into smart contract security analysis will become increasingly important in protecting blockchain ecosystems from future risks.

G. AGENTIC AI

Agentic AI refers to autonomous, intelligent agents capable of decision-making, self-learning, and task execution with minimal human involvement. This emerging paradigm [25] holds substantial promise in enhancing vulnerability detection and resolution in smart contracts by automating critical security tasks. Agentic AI can autonomously perform vulnerability identification, conduct dynamic behavior analysis, and support real-time monitoring of smart contracts throughout their lifecycle. By continuously analyzing smart contract source code and bytecode, leveraging advanced techniques such as symbolic execution and fuzz testing, agentic AI can uncover vulnerabilities like reentrancy, integer overflows, and access control misconfigurations without manual intervention.

Through the integration of machine learning, these agents can learn from historical vulnerability data, improving detection accuracy over time. They can simulate contract behavior under diverse execution scenarios to detect runtime anomalies and flag suspicious interactions in real time. In advanced implementations, agentic AI can autonomously apply patches or roll back unsafe deployments. By incorporating natural language processing (NLP), they can also parse and interpret human-readable contract clauses to better understand logic and intent, enabling deeper semantic validation. Additionally, their ability to conduct autonomous penetration testing and interoperate with decentralized networks makes them highly adaptable to evolving threat landscapes. As a result, agentic AI streamlines the end-to-end process of vulnerability detection, mitigation, and continuous improvement, significantly bolstering the security and reliability of blockchain-based smart contracts.

In the broader DevSecOps context, agentic AI serves as a catalyst for intelligent automation across the development, security, and operations pipeline. Unlike conventional AI systems, which act in response to pre-programmed instructions, agentic AI actively monitors, assesses, and responds

to dynamic software environments. These agents engage in continuous analysis of logs, telemetry data, code repositories, and infrastructure layers to identify anomalous behaviors and emerging threats. They automatically correlate signals across multiple layers to filter out noise, reduce false positives, and prioritize high-impact issues.

Moreover, agentic AI agents integrate the outputs of static application security testing (SAST), dynamic analysis (DAST), software composition analysis (SCA), and Infrastructure as Code (IaC) security tools. They intelligently rank vulnerabilities based on contextual factors such as exploitability, business impact, and real-time threat intelligence, and can automatically create and assign remediation tasks within platforms like Jira or GitLab, complete with actionable, context-aware suggestions. With direct access to source code and configuration files, these agents can propose and apply secure code changes, ranging from patching insecure functions to refactoring misconfigured Kubernetes or Terraform manifests, all within sandboxed environments for review.

They also bring adaptability to CI/CD pipelines, dynamically adjusting pipeline behavior in response to contextual signals, such as tightening security gates for high-risk changes, triggering additional scans for sensitive components, or halting releases when critical issues are detected. These decisions are not static but evolve over time, allowing for continuous refinement of security workflows based on real-world feedback.

In terms of compliance, agentic AI plays a key role in ensuring continuous adherence to regulatory standards like ISO 27001, SOC 2, NIST, and GDPR. It maps CI/CD activities to specific compliance controls, generates audit-ready evidence automatically, and flags deviations in real time. For developers, these agents offer embedded security coaching, providing real-time alerts about insecure coding patterns within integrated development environments (IDEs), explaining vulnerabilities in understandable terms, and recommending secure coding alternatives. Personalized training resources can also be generated based on individual or team coding behaviors.

By learning from incident reports, red team engagements, and developer feedback loops, agentic AI continually refines its threat models and decision-making algorithms. It can collaborate with other AI agents across systems, facilitating orchestrated, layered defense strategies. Acting as a unifying layer across toolchains, spanning code repositories, CI/CD systems, security scanners, cloud platforms, and observability stacks, agentic AI ensures that security remains a coordinated and continuous effort.

In summary, agentic AI redefines DevSecOps as a self-sustaining, proactive, and intelligent ecosystem. It accelerates threat detection, automates resolution, enhances team collaboration, and fosters a culture of secure-by-design development. As this technology matures, it is expected to play a pivotal role in scaling secure software delivery across complex and fast-moving digital environments, particularly

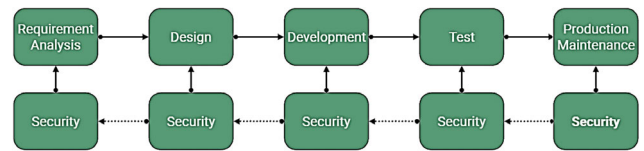


FIGURE 2. Shift-left security strategy.

in the context of blockchain, smart contracts, and other decentralized applications.

V. DEVSECOPS PRINCIPLES

Each year, the number of vulnerabilities discovered in production smart contracts continues to rise, posing significant risks to both customers and businesses. Smart contract-based projects, due to their high-profile nature and considerable financial value, attract attackers seeking to exploit weaknesses in security. Writing secure code from the outset and addressing vulnerabilities after release, especially following a successful attack on the blockchain, is a complex and resource-intensive challenge. In this context, DevSecOps [7] plays a pivotal role in blockchain development by integrating security measures early in the software development lifecycle. This approach, referred to as the shift-left security strategy, ensures that security is built into the development process from the very beginning, helping to identify vulnerabilities before they can be exploited. As depicted in Figure 1, DevSecOps advocates for proactive security practices, while Figure 2 further illustrates the continuous embedding of security measures throughout the development lifecycle.

Applying the shift-left strategy to detect vulnerabilities in smart contracts shifts the focus from post-deployment fixes to integrating security practices throughout the development process. This proactive approach involves embedding security considerations into the design, development, and testing phases, continuously testing the code for vulnerabilities before deployment. Here's how the shift-left approach can be effectively applied to smart contract vulnerability detection:

- **Early Security Requirements:** Before any code is written, it is essential to define security requirements, ensuring that the contract is designed with secure practices from the start. This involves specifying the types of vulnerabilities the contract must be protected against, such as reentrancy attacks, integer overflows, and access control issues, and establishing the necessary protections for each vulnerability.
- **Code Analysis During Development:** As developers write the smart contract, they should incorporate static analysis tools, such as Slither or Mythril, into the development pipeline to automatically detect common vulnerabilities like integer overflows, reentrancy, and unauthorized access. These tools can be configured to scan the code in real-time, identifying issues as they arise and allowing for rapid resolution.

- **Real-time Threat Modeling:** Developers can also employ threat modeling techniques early in the development process to anticipate vulnerabilities within the contract's logic. This includes assessing how the contract interacts with external systems (such as oracles or other contracts) and pinpointing potential attack vectors, such as front-running or denial-of-service (DoS) vulnerabilities. Early threat modeling ensures that the contract design anticipates and mitigates common pitfalls.
- **Automated Unit Testing and Fuzzing:** Unit tests should be written for each component of the smart contract to verify that each function behaves as intended. Additionally, fuzz testing tools, such as ContractFuzzer, can be used to generate a wide range of inputs that might expose vulnerabilities in the contract. By incorporating fuzz testing early in the development process, developers can detect issues in a controlled environment before deployment.
- **Formal Verification:** Integrating formal verification tools like KEVM or ZEUS ensures that the smart contract's behavior is mathematically proven to adhere to specific security properties. Formal verification tools analyze the contract's logic in-depth, identifying subtle vulnerabilities such as incorrect logic or flawed access control before they can be exploited, significantly reducing the likelihood of post-deployment issues.
- **Security Reviews and Peer Audits:** Regular code reviews and external security audits are essential to identifying vulnerabilities that automated tools may miss. Peer reviews should be conducted early and often, ensuring that the contract adheres to best practices and established security standards. Independent security experts can provide valuable insights that enhance the contract's security posture.
- **Integration Testing:** Smart contracts often interact with other contracts or external systems, and integration testing is necessary to ensure these interactions are secure. This phase should involve simulating potential attack vectors and testing the contract's response to various scenarios, such as cross-contract attacks, to ensure that no vulnerabilities are exposed during interaction.
- **Continuous Monitoring and Deployment:** Once deployed, real-time monitoring tools can track the performance of the smart contract and detect any anomalies that may indicate malicious behavior or vulnerabilities. Continuous integration pipelines should be configured to rapidly apply patches when new vulnerabilities are detected, enabling quick mitigation and reducing the risk of exploitation.

By shifting security practices left, vulnerabilities in smart contracts can be detected and addressed early, reducing the likelihood of successful attacks and avoiding costly post-deployment fixes. This proactive approach ensures that smart contracts are built with a solid security foundation, ultimately fostering trust and safety in decentralized applications and blockchain ecosystems.

In the broader scope of DevSecOps, security activities span the entire software development lifecycle, from the initial design and development stages to ongoing operational security. During the development phase, the focus is on identifying and preventing vulnerabilities before deployment. Once the contract is live, operational security ensures that the system is continuously monitored and defended against emerging threats in real-time. To effectively address blockchain security vulnerabilities, it is crucial for developers to adopt DevSecOps practices and update their development pipelines accordingly. By implementing the strategies outlined in Table 3, organizations can create a robust DevSecOps framework, ensuring that security remains a continuous, integral part of the smart contract lifecycle. This approach not only mitigates risks but also streamlines vulnerability detection and allows for the rapid application of patches, preventing costly attacks and improving the overall security of blockchain systems.

VI. DEVSECOPS PHASES

In Figure 3, the foundational stages of the DevSecOps pipeline are illustrated in conjunction with integrated security activities that span the entire Continuous Integration/Continuous Deployment (CI/CD) process. This structure embodies the core DevSecOps philosophy of “security as code”, embedding security controls and verification into each step of the software development lifecycle, from initial coding to production deployment.

The process begins with source code preprocessing, compilation, and building, during which static application security testing (SAST) tools are applied to identify insecure coding practices, hardcoded credentials, and vulnerable dependencies. These early-stage tools provide rapid feedback to developers, allowing issues to be remediated before they propagate further downstream. Once the application is built into deployable artifacts, a suite of automated test frameworks is triggered, covering unit, integration, and regression tests to validate core functionality and performance. Security-centric tests, such as dynamic application security testing (DAST), fuzz testing, and software composition analysis (SCA), are also run to uncover runtime vulnerabilities and threats in third-party libraries or open-source components.

A critical aspect of this stage is the generation of automated security evidence reports, which consolidate results from all security scanning and validation tools. These reports serve as living documentation of an application's security posture and are essential for regulatory compliance, incident response planning, and fostering transparency across development, operations, and security teams.

As the pipeline advances, the software enters the release and deployment phases, where it is promoted to staging and production environments using secure, automated workflows. Tools such as Infrastructure as Code (IaC) (e.g., Terraform) and configuration management systems (e.g., Ansible, Puppet) ensure that infrastructure is provisioned and maintained according to security policies and organizational standards.

TABLE 3. DevSecOps principles in smart contract.

Principle	Description
Empowering engineering teams	Security should not be the sole responsibility of specialized security personnel. Instead, engineering teams must be equipped with the tools, knowledge, and guidance needed to take ownership of security in their development efforts. This can be achieved by embedding security experts within teams to provide mentorship and by fostering a culture of knowledge sharing through training, documentation, and regular security reviews. When every team member understands and participates in secure development practices, the organization builds a stronger first line of defense against threats.
Increasing the visibility of security	To make security a tangible part of daily operations, it must be visible and measurable. Security issues should be tracked like any other engineering work item, via tickets, dashboards, and metrics. This visibility helps align security goals with business objectives and allows stakeholders to understand the risk landscape. Making security data accessible across teams ensures that decisions are made with security impact in mind, fostering accountability and transparency.
Shift left	The "shift-left" philosophy emphasizes integrating security early in the development lifecycle, starting from requirements gathering, through design, implementation, and testing. Instead of addressing vulnerabilities post-deployment, developers are encouraged to identify and remediate issues during the early stages, reducing cost and risk. Continuous feedback loops at every stage, from development to production, help refine security practices and ensure ongoing improvement.
Integrating security in CI/CD	With the shift-left approach, CI/CD workflows evolve beyond building and deploying applications, they also incorporate automated security scanning, static and dynamic analysis, and dependency checks. Security tools such as Snyk, SonarQube, and OWASP Dependency-Check are integrated into the CI/CD pipeline to detect vulnerabilities in real time and provide actionable remediation steps. This automation accelerates the development process while ensuring security is not compromised.
Continuous security	Security must be treated as an ongoing, dynamic process rather than a one-time activity. Continuous security entails embedding threat detection, code scanning, and policy enforcement as recurring actions within both development and operations workflows. This ensures that new threats are addressed as they emerge, and that security becomes a sustainable part of the DevOps culture.
Continuous monitoring	By implementing continuous monitoring, teams gain real-time insights into the behavior and security posture of deployed applications. Security information and event management (SIEM) systems, anomaly detection tools, and log analysis platforms provide alerts for suspicious activity. This feedback loop allows development teams to respond rapidly to emerging threats and adapt their code accordingly.
Continuous configuration	Automated configuration management ensures consistency, security, and compliance across both software and infrastructure layers. It covers everything from application logic to environment setup, provisioning, and patching of physical and virtual components. Tools like Ansible, Terraform, and Puppet can enforce secure baselines and reduce misconfigurations, which are a major source of vulnerabilities.
Preventing and protecting	A comprehensive DevSecOps strategy combines secure coding practices during development with real-time protections during operations. This dual approach includes input validation, access control, and error handling in code, along with runtime application self-protection (RASP) and intrusion detection systems (IDS) in production. By preventing vulnerabilities early and protecting against them at runtime, teams create a resilient security posture that spans the entire lifecycle of the application.
Security by design	Security by Design promotes embedding security into every layer of a system from the start, beginning at the architectural level with secure defaults, clear trust boundaries, and minimal attack surfaces. It includes controls like input validation, access control, and encryption, as well as early threat modeling to proactively mitigate risks. By integrating security throughout the development lifecycle, this approach builds more resilient systems, reduces post-deployment vulnerabilities, and supports compliance, making it a key component of modern DevSecOps.

Secure artifact repositories and version control mechanisms further enhance traceability and reproducibility.

Runtime monitoring is then initiated, leveraging advanced observability tools to track application performance, detect

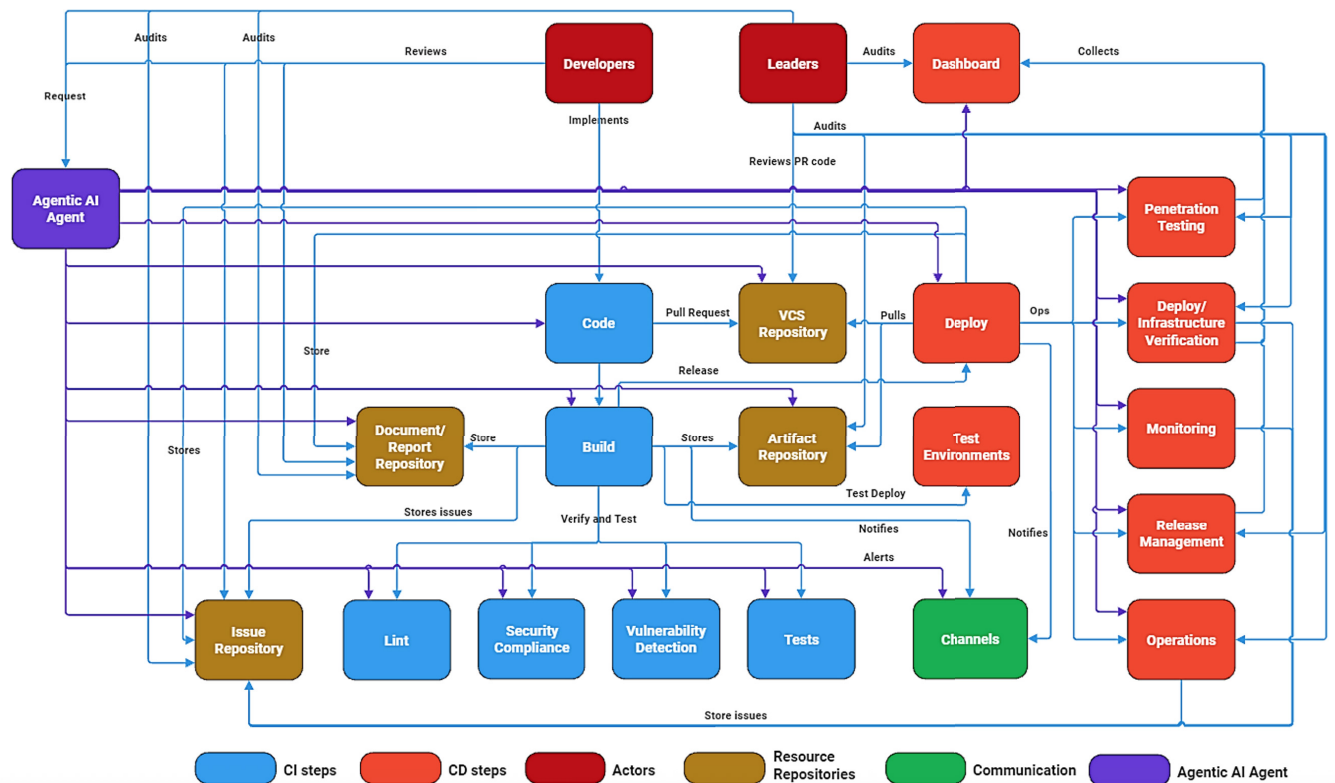


FIGURE 3. DevSecOps development.

anomalies, and respond to suspicious behaviors in real-time. Security Information and Event Management (SIEM) systems and runtime protection mechanisms ensure continuous surveillance and fast threat mitigation.

Importantly, Agentic AI agents [25], [63] are integrated throughout the pipeline to provide intelligent, autonomous oversight. These agents continuously monitor dashboards, source control systems, issue repositories, and security reports. Upon detecting anomalies, misconfigurations, or critical vulnerabilities, they generate alerts, recommend remediation actions, or even trigger automated responses, further enhancing the agility and responsiveness of the security posture.

This comprehensive integration of security across the DevSecOps pipeline not only facilitates the early and continuous detection of vulnerabilities but also accelerates resolution, reduces human error, and enforces a culture of “secure-by-default” development. By transforming security from a peripheral function into a shared, embedded responsibility, DevSecOps fosters collaboration across disciplines and enables the rapid delivery of secure, reliable, and resilient software systems.

A. CONTINUOUS INTEGRATION (CI)

The primary objectives of Continuous Integration (CI) in smart contract development are to accelerate the detection and resolution of code and security issues, while minimizing

the time required for testing and deploying updates. CI fosters a streamlined, automated process in which developers can frequently push changes to a shared repository. Each update is automatically validated through a build pipeline, which includes a suite of tests designed to uncover both functional bugs and security vulnerabilities early in the development lifecycle.

In this workflow, developers can make local changes and run tests to verify functionality. Once the local tests pass, the changes are committed and submitted via a merge request (MR). This MR then triggers the CI pipeline, which executes automated processes such as compilation, unit testing, static analysis, and security scans. Only after all automated checks pass can the code be merged into the main development branch. This ensures that only validated and secure code progresses through the pipeline, thereby reducing the risk of introducing defects or vulnerabilities into production.

CI pipelines are typically executed for every commit or merge request, ensuring continuous quality control. Popular CI/CD platforms like Jenkins, Travis CI, GitLab CI/CD, and GitHub Actions provide robust frameworks to automate this process. These platforms can be extended with smart contract-specific tools such as Truffle, Hardhat, Embark, Brownie, and Waffle. These development frameworks include features like local blockchain environments, contract deployment scripts, and integrated testing utilities, making them ideal for automated smart contract validation.

By integrating these tools into CI pipelines, teams can automate the compilation, deployment, testing, and debugging of smart contracts across various stages. For example, Hardhat can simulate a local Ethereum network, enabling fast test execution and detailed error reporting. Brownie, on the other hand, supports Python-based testing and is well-suited for integration with security analysis tools. These frameworks are easy to install, configure, and integrate into CI pipelines, ensuring that every code change is thoroughly vetted before it reaches production. This automated, repeatable process not only enhances code quality and security but also aligns with DevSecOps best practices for continuous delivery and safe deployment in blockchain ecosystems.

1) CODE

The code phase of smart contract development centers around the creation and refinement of core functionalities, supported by robust Integrated Development Environments (IDEs) and specialized frameworks. A prominent tool in this phase is the Remix IDE, which is widely used for writing and testing Solidity smart contracts. Remix provides an all-in-one environment featuring an integrated compiler, a Solidity runtime, and a variety of developer-friendly tools that streamline the development lifecycle.

To accelerate development and ensure code quality, Remix offers features such as contract verification, static analysis (linting), automatic documentation generation, debugging, deployment, and unit testing. These tools collectively reduce development complexity and help developers identify bugs or vulnerabilities early. The overarching design principle in this phase is to simplify smart contract engineering by minimizing cognitive overhead and focusing on modular, well-scoped contract logic.

To optimize performance and reduce gas consumption, developers are encouraged to design contracts that minimize on-chain transaction size by focusing each contract on a single responsibility or functionality. Additionally, using battle-tested libraries such as OpenZeppelin is highly recommended. OpenZeppelin provides secure, community-audited implementations of commonly used standards like ERC-20, ERC-721, and AccessControl, which reduces the risk of introducing vulnerabilities.

Best practices during the coding phase also include:

- Locking library and compiler versions to prevent unexpected behavior from future updates or compatibility issues.
- Writing comprehensive inline documentation using the NatSpec format, which helps clearly describe the purpose, parameters, return values, and side effects of contract functions.
- Following standard naming conventions, applying access modifiers appropriately, and avoiding patterns known to lead to vulnerabilities (e.g., unchecked external calls or improper fallback functions).

By adhering to these principles and leveraging powerful IDE features, teams can build smart contracts that are secure, maintainable, and aligned with DevSecOps practices, laying a strong foundation for the subsequent testing, integration, and deployment phases.

2) BUILD

The build phase in smart contract development is responsible for transforming source code into deployable artifacts, such as EVM bytecode and ABIs (Application Binary Interfaces). This process often involves several preparatory and compilation steps that ensure the code is production-ready and compatible with the Ethereum Virtual Machine (EVM).

One key step in this phase is the optional use of a preprocessor, such as Solpp, which streamlines Solidity development by processing symbols, macros, and expressions, and resolving file system imports. Solpp can merge all referenced dependencies into a single consolidated file, simplifying compilation and reducing potential issues with file resolution. It is a best practice to maintain a clear separation between raw source files and generated output, storing original files in a dedicated directory while using the preprocessor to populate the source directory used by the CI/CD pipeline.

For more complex or automated build workflows, developers often leverage the JSON interface for compilation. This method allows the Solidity compiler to accept input and return output in standardized JSON format, making it easier to integrate into build systems and subsequent tooling stages (e.g., testing or deployment scripts).

Two commonly used compilers are:

- Solc: A native C++ implementation of the Solidity compiler, optimized for performance.
- Solc-js: A JavaScript version of Solc, compiled via Emscripten, making it suitable for browser-based or Node.js environments.

Both compilers translate Solidity source code into EVM-compatible bytecode and generate the ABI, which defines the contract's interface and is essential for interacting with the deployed contract programmatically.

By structuring the build phase in this way, incorporating preprocessing, standardized JSON compilation, and the generation of essential artifacts, teams can create reliable and repeatable builds, a critical requirement for any secure and scalable DevSecOps workflow in blockchain development.

3) TEST

After a successful compilation, the smart contract application is immediately deployed to a designated test environment where it undergoes automated testing to ensure functionality, correctness, and security. This step is integral to the continuous integration (CI) process, enabling rapid feedback and early detection of potential issues before deployment to production.

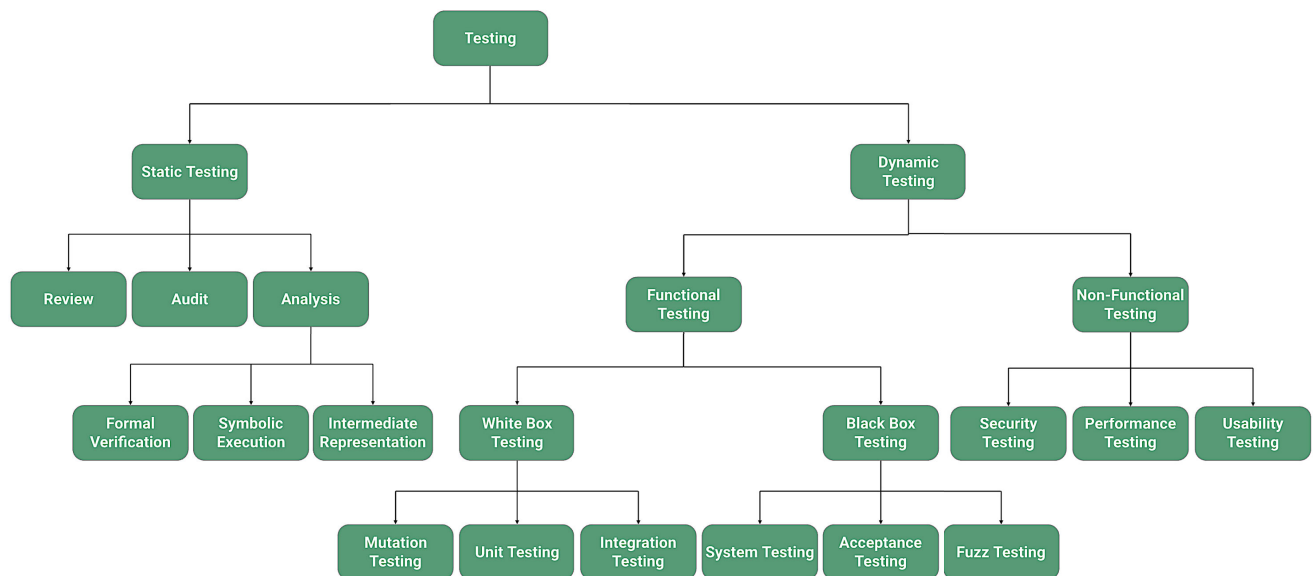


FIGURE 4. Smart contract testing methods.

As illustrated in Figure 4, the testing phase encompasses a range of test categories, each targeting different aspects of the contract's behavior and resilience. These include unit tests, integration tests, fuzz tests, property-based tests, and security-specific analyses. Each category plays a distinct role in validating the contract, from checking individual functions to simulating adversarial conditions.

The CI environment leverages well-established testing tools and frameworks, such as Truffle, Hardhat, Brownie, and Foundry, that provide built-in support for test automation and simulate blockchain environments for realistic testing conditions. These tools also allow the integration of static analyzers and fuzzers to further strengthen the security posture of the contracts.

Furthermore, Table 1 presents a detailed classification of various vulnerability detection techniques, organized by their corresponding test types. This structured approach helps development teams select the most suitable methods for identifying issues ranging from logic errors to critical security flaws.

By embedding rigorous and automated testing into the CI pipeline, developers can ensure that smart contracts are not only functional but also secure, efficient, and resilient against potential attacks, aligning with DevSecOps best practices for proactive, continuous security assurance.

α: TESTING ENVIRONMENTS

To ensure the effectiveness and reliability of smart contracts, it is crucial to test them within an existing blockchain ecosystem, whether it is a permanent blockchain or a temporary test environment specifically designed for development purposes. A common tool for such testing is Ganache, which acts as a local blockchain emulator for testing and

development. Ganache simulates the characteristics of a real blockchain network, providing a controlled and customizable environment for comprehensive testing and analysis.

The primary advantage of using a local blockchain like Ganache is its flexibility. Developers can manipulate key parameters such as gas price, mining speed, and time flow, allowing automated tests to run at a faster pace and reducing both time and cost constraints. This flexibility makes it an ideal tool for accelerating the development cycle. The following methods can be employed to adequately prepare a blockchain for testing:

- **Empty Blockchain Setup:** Initially, a blank blockchain, free of any prior transaction records, can be used to test operations that are independent of transaction history. This serves as the first stage in many testing scenarios, offering a clean slate for various use cases.
- **Artificial Blockchain Generation:** Specialized tools can be used to generate a populated blockchain by creating transactions and forming valid blocks. This approach allows for the simulation of more complex blockchain activities and interactions, providing a realistic testing environment.
- **Blockchain Snapshot:** Another efficient option is to create a snapshot of the blockchain's current state. By capturing the state of the blockchain at a particular block, developers can revert the chain back to that specific snapshot whenever needed, simplifying testing and ensuring repeatable test conditions.
- **Forking a Production Blockchain:** In cases where replicating a live blockchain is necessary, the blockchain can be forked. A local development chain can be created that mirrors the state of the production blockchain, with the local chain managing new blocks while the

forked chain handles the tasks related to older blocks. This allows for testing in an environment that is almost identical to production while maintaining the integrity of the original blockchain.

By utilizing these strategies, developers can effectively simulate real-world conditions and rigorously test their smart contracts, ensuring they function as expected before deployment to production.

b: TESTING METHODS

Dynamic analysis involves testing and assessing an application in real-time to perform dynamic code scanning, bug diagnosis, and memory issue resolution. This method is essential for ensuring that smart contracts behave as expected according to predefined test cases, through unit and integration tests. As noted by Chakraborty et al. [40], unit tests play a critical role in blockchain technology to verify contract accuracy. These unit tests should cover all contract methods or, at the very least, the publicly accessible ones. To ensure reliability, the tests must validate that only valid input parameters are accepted and that the expected outcomes are achieved. Additionally, tools like Truffle assertions and OpenZeppelin Test Helpers can assist in validating event emissions, execution rollbacks, and role-based access control by providing transaction receipts or logs for better verification.

A key aspect of unit tests is their independence from one another, meaning each test should operate in isolation. This ensures that every test is treated as a separate entity, contributing to a more reliable and comprehensive testing environment. Integration tests, on the other hand, evaluate complex scenarios involving multiple contract calls across various contracts. These tests are crucial for assessing the interoperability and functionality of the system, covering interactions between smart contracts and client applications. Blockchain development frameworks can include various test cases focused on smart contract interactions within the blockchain, while other integration testing tools focus on testing interactions between contracts and external client applications.

Solidity, the programming language for writing smart contracts, allows for low-level testing of every function within a contract, whereas JavaScript tests tend to focus on external and public function behaviors. To facilitate asynchronous testing, the Mocha testing framework, in conjunction with Chai (an assertion library), is often used for smart contracts. Some frameworks like Waffle and MockContract dynamically create mock objects within the test code, while tools such as OpenZeppelin and Truffle enable parallel test execution to speed up the testing process.

A widely adopted technique in dynamic testing for vulnerability detection is fuzzing, where a large number of random inputs are fed to the system to monitor for abnormal behavior. One such fuzzing-based method for detecting security vulnerabilities in Ethereum smart contracts

is ContractFuzzer [13], which utilizes the smart contract's ABI specification to generate fuzzing inputs and create an effective test plan for uncovering potential weaknesses.

While dynamic testing is crucial, static analysis is also a commonly used approach to scrutinize smart contract code without execution. Tools like linters help improve code quality by identifying minor issues, ensuring best practices, and flagging syntax errors or style guideline violations. In addition, more advanced tools for detecting security vulnerabilities employ techniques such as formal verification, symbolic execution, and deep learning. Prominent tools include ZEUS [33], KEVM [15], Oyente [8], and Maian [9], which are designed to uncover complex vulnerabilities that may be missed during dynamic analysis.

c: TESTING COVERAGE

Test coverage refers to the measure of how thoroughly an application's code has been tested, ensuring that all code paths, functions, and features are appropriately validated by the test cases. The goal is to identify potential bugs, logic errors, and vulnerabilities before deployment. Comprehensive test coverage significantly reduces the likelihood of encountering unforeseen issues during the application's operation, especially when the test outcomes align with the expected results. This leads to greater confidence in the application's reliability and security, particularly for critical applications like smart contracts.

However, performing test coverage within an Ethereum Virtual Machine (EVM) environment comes with certain trade-offs. Specifically, enabling test coverage can result in increased gas consumption and slower test execution times, as each code path or transaction involves additional overhead for tracking coverage data. This can be particularly problematic for larger projects, where excessive gas costs and prolonged test times can hinder development efficiency and inflate testing expenses.

To mitigate these challenges, it is considered best practice to run test coverage in a separate Continuous Integration (CI) pipeline. By isolating test coverage jobs from the regular testing workflow, developers can ensure that the main testing process remains efficient and that coverage tracking does not interfere with the core development cycle. This separation allows for the optimization of gas consumption and the minimization of testing delays.

Tools like Sol-coverage and eth-gas-reporter are commonly used to generate detailed reports that measure the coverage percentage and track gas consumption during testing. These tools offer valuable insights into the effectiveness of test cases, highlighting areas of the code that may be inadequately tested, and help identify opportunities for optimization. In addition, the reports provide developers with a clear understanding of the associated costs, including the gas used during tests, enabling more informed decisions regarding resource allocation.

In summary, while test coverage is a critical part of ensuring robust and secure smart contracts, developers must

balance thorough testing with performance considerations. By using dedicated CI jobs and coverage tools, teams can enhance testing efficiency, improve code reliability, and manage the costs associated with testing smart contracts.

B. CONTINUOUS DEPLOYMENT (CD)

Continuous Deployment (CD) is a software development methodology that automates the process of preparing and deploying code modifications to production environments. This approach streamlines the release cycle, ensuring that new features, updates, or bug fixes are delivered quickly and efficiently. In traditional software development, deployment pipelines are essential for pushing code and configurations to various hosting environments, such as virtual machines, containers, or serverless functions. These environments can be provisioned using an Infrastructure-as-Code (IaC) approach, where configuration files are used to automate the setup and management of infrastructure resources.

In the context of blockchain development, however, deployment pipelines take on a slightly different role. Instead of simply deploying code to server environments, these pipelines are responsible for the deployment of essential blockchain components, such as smart contracts, decentralized applications (dApps), and related configurations, across various blockchain networks or environments.

Blockchain deployment pipelines typically involve multiple steps:

- **Smart Contract Deployment:** This includes the process of deploying compiled smart contracts to the blockchain network (e.g., Ethereum, Binance Smart Chain). It may involve setting up the contract on testnets for initial validation and then moving to production once the contract passes all tests.
- **Application Deployment:** Alongside smart contracts, the associated decentralized applications (dApps) or front-end services need to be deployed. These dApps interact with the blockchain to execute contract functions and display results to the users.
- **Network Configuration and Management:** Blockchain networks, whether public or private, require proper setup and configuration, including node management, consensus mechanism configuration, and connecting dApps with the correct blockchain nodes.
- **Automated Testing:** Just as traditional CD pipelines include automated tests, blockchain deployment pipelines must include testing for smart contracts and dApps. This includes unit tests, integration tests, and security audits to ensure that deployed contracts perform as expected and are secure.
- **Monitoring and Maintenance:** After deployment, continuous monitoring of both the blockchain network and the deployed applications is crucial. Tools for performance tracking, transaction monitoring, and auditing smart contracts help detect issues early and ensure the system runs smoothly.

By incorporating blockchain-specific elements into a CD pipeline, development teams can ensure that updates to smart contracts and dApps (decentralized Apps) are deployed efficiently, securely, and consistently, aligning with best practices and minimizing downtime. This deployment methodology significantly enhances the agility and reliability of blockchain-based applications.

1) RELEASE

Once a build successfully passes a series of manual and automated tests, it progresses to the release stage, where it is prepared for deployment in the production environment. At this stage, all critical deployment elements, including metadata, Application Binary Interface (ABI), and bytecode, must be securely stored in a shared environment with appropriate access controls and permissions to ensure that only authorized personnel can access and modify these components.

In the case of smart contracts, which often manage sensitive operations and significant capital, it is essential to conduct independent security audits before deployment. These audits thoroughly assess the smart contracts for vulnerabilities, coding errors, and compliance with best security practices. Auditing becomes even more critical for contracts in high-stakes areas, such as finance, healthcare, and supply chain management, where any compromise can have severe consequences.

Furthermore, within the release phase, it is advisable to implement a manual approval process for granting deployment to production. This step provides an additional layer of control and oversight. Designating specific individuals within the organization, such as senior developers, security officers, or team leads, ensures that the decision to release into production is reviewed by trusted parties. This procedure adds accountability and minimizes the risk of accidental or unauthorized deployments.

2) DEPLOY

The deployment phase is responsible for overseeing the entire process of moving finalized builds into a live environment. This phase involves managing and coordinating the transition from testing to production, ensuring that the application is reliably and securely delivered to end users. To streamline this process, various automation measures are employed, reducing manual intervention and minimizing the chances of human error. Automated deployment pipelines ensure that the process is consistent, repeatable, and efficient.

In addition to automation, best practices are followed to enhance the dependability and security of the release procedures. These include implementing robust monitoring, rollback strategies, and post-deployment validation checks. By automating tasks like provisioning infrastructure, deploying smart contracts, and updating relevant system components, the deployment phase becomes more predictable, reducing the time to production and increasing the overall reliability of the application.

a: IAC (INFRASTRUCTURE AS CODE)

Infrastructure as Code (IaC) refers to the process of automating the deployment and provisioning of system environments through code or configuration files. The primary goal of IaC is to ensure the consistency and repeatability of system components, configurations, and topology. By using IaC configuration files, organizations can manage infrastructure as easily as application code, allowing for easier modifications, version control, and distribution of configurations. This approach is especially valuable in environments like blockchain systems, where consistent and error-free provisioning is crucial. It helps prevent manual errors and reduces time and resource expenditure when managing complex blockchain environments.

Several IaC tools, such as Terraform, Ansible, Puppet, and Chef, can be used to automate the deployment and provisioning of system environments. These tools enable infrastructure management to be integrated into continuous integration/continuous deployment (CI/CD) pipelines, enhancing automation, scalability, and reliability. By adopting IaC, organizations streamline their infrastructure management processes, enabling quicker provisioning and more stable system configurations.

In parallel with IaC, Blockchain-as-a-Service (BaaS) offers a cloud-based platform for businesses to create and manage blockchain networks. BaaS allows organizations to leverage blockchain technology without having to manage the underlying infrastructure themselves, reducing complexity and operational costs. Through third-party services, companies can build, host, and operate their blockchain applications with ease. A prominent example of BaaS is Microsoft Azure's partnership with ConsenSys to offer Ethereum Blockchain-as-a-Service. This service enables companies to deploy and manage Ethereum-based blockchain solutions within a secure, scalable cloud environment. Similarly, AWS Managed Blockchain provides BaaS solutions based on open-source frameworks like Ethereum and Hyperledger Fabric, empowering businesses to develop decentralized applications without the need to maintain blockchain infrastructure themselves. These BaaS solutions, combined with IaC tools, offer businesses a comprehensive, efficient, and scalable approach to integrating blockchain technology.

b: DEPLOY SMART CONTRACTS

Deploying a smart contract requires ensuring that all necessary libraries and dependent contracts are properly deployed and available. It is not advisable to hard-code dependency details directly in the contract, particularly for testing purposes. Instead, tools such as Truffle Migrations or Hardhat Ignition can be used to automate the process of linking contracts and populating them with initial data. This ensures a more streamlined deployment process that is easier to manage.

Upgrading a smart contract typically requires deploying a new version of the contract. When this occurs, client

applications must be updated to reflect the new contract address, and the state data from the old contract must be manually transferred to the new version. However, this manual process can be cumbersome and prone to errors. To simplify this, the proxy pattern, specifically Contract Relay [41], can be employed. This pattern allows for contract implementation to be replaced without changing the contract's address, state, or balance. By using delegate-call mechanisms, functions from different contracts can be executed as if they were part of the calling contract, enabling users to interact with a proxy contract that manages the state and delegates transactions to other contracts with relevant functionalities.

To avoid conflicts in scope and storage between the proxy and logic contracts, state variables must be handled carefully using design patterns like Inherited, Eternal, and Unstructured Storage [42]. These patterns provide mechanisms to separate storage logic from the proxy contract, preventing issues in state management during contract upgrades.

Furthermore, Ethereum Improvement Proposal (EIP) 2535, also known as Diamonds, Multi-Facet Proxy, establishes a standard for creating modular smart contract systems that can undergo seamless upgrades. By integrating standard plugins, such as those provided by OpenZeppelin, into development environments, developers can more efficiently deploy and manage upgradeable smart contracts. This integration simplifies the upgrade process, ensuring that contracts remain maintainable, secure, and flexible over time.

c: TestNets

To ensure their reliability, functionality, and performance, smart contracts need to undergo rigorous testing on a public test network, commonly referred to as a Testnet, before being deployed on the Mainnets. Testnets provide an environment that mirrors the real blockchain, allowing developers to test their contracts without risking real assets. A widely used Testnet in the Ethereum ecosystem is Goerli [43], which operates on Ethereum and utilizes a cross-client Proof-of-Authority (PoA) consensus mechanism. Goerli boasts a block time of approximately 15 seconds, ensuring efficient transaction processing and enabling developers to quickly validate and refine their contracts.

Another notable Ethereum Testnet is Ropsten, which employs a cross-client Proof-of-Work (PoW) consensus mechanism. Ropsten has a block time of less than 30 seconds, ensuring timely validation of transactions, making it a reliable option for testing Ethereum smart contracts under conditions similar to the Mainnet. However, while Ropsten offers the benefit of simulating a PoW environment, it also presents some challenges. PoW Testnets, including Ropsten, are known for experiencing more unpredictable block times, higher network congestion, and frequent chain reorganizations. These characteristics can make testing less reliable and sometimes result in inconsistent transaction behaviors.

In contrast, PoA Testnets like Goerli are generally considered to be more stable and predictable, as they do not rely on the computationally intensive Proof-of-Work mechanism. The PoA consensus allows for quicker and more consistent block finality, reducing the likelihood of chain forks and enabling more stable testing conditions. As a result, PoA Testnets like Goerli are often preferred for tasks that require more reliable and consistent network performance.

Overall, choosing the right Testnet for deployment and testing depends on the specific needs of the contract and the desired testing conditions. While Goerli provides a stable and efficient environment for smart contract testing, Ropsten may be useful for testing under a PoW-like consensus but requires caution due to its inherent instability.

d: MainNets

When deploying smart contracts on MainNets, the process involves ensuring that the contracts are thoroughly tested and verified on TestNets before their release to the live environment. The deployment to MainNets is the final stage of the smart contract lifecycle, where the code is pushed to a public blockchain that interacts with real users and assets. Here's how MainNets are applied in deploying smart contracts:

- **Pre-Deployment Testing on TestNets:** Before deploying on MainNets, smart contracts undergo extensive testing on Ethereum TestNets like Goerli [43]. These TestNets simulate the conditions of the MainNet but use test Ether and do not affect real assets, making them ideal for testing contract functionality and ensuring reliability.
- **Smart Contract Verification:** Once the contract is tested on TestNets, it is necessary to verify that the deployed contract matches the intended source code. This is done using blockchain explorers like Etherscan [44] or Sourcify [45] for both TestNets and MainNets. Tools like hardhat-etherscan automate this verification process within the Continuous Deployment (CD) pipeline, ensuring that the deployed smart contract on the MainNet matches the source code uploaded.
- **Deployment to MainNet:** After successful testing and verification on TestNets, the contract is deployed to a MainNet such as Ethereum MainNet or Binance Smart Chain (BSC). This step is often carried out using deployment tools such as Truffle Migration, which automates the deployment process to MainNets and ensures the contract is live on the blockchain for real-world interactions.
- **Smart Contract Interaction on MainNet:** Once deployed, users and applications can interact with the smart contract on the MainNet. Since MainNets handle real assets and transactions, it is essential to ensure that the contract behaves as expected, particularly in terms of transaction processing, gas fees, and contract security.
- **Continuous Monitoring and Analytics:** After deployment, it is crucial to monitor the performance and

health of the contract on the MainNet. Services like Etherscan [44] and AWS Amazon Managed Blockchain provide real-time monitoring tools to track on-chain data such as transactions, gas usage, contract status, and more. These tools help identify potential issues early and maintain the contract's operational efficiency.

By applying MainNets for deploying smart contracts, developers ensure that their contracts operate in a live environment with real data and assets. The deployment process involves a careful sequence of testing, verification, deployment, and ongoing monitoring to ensure that the smart contract remains secure, reliable, and functional for its users.

C. OPERATE AND MONITOR

When deploying smart contracts on a permissionless blockchain, it is often necessary to publicly verify that the deployed contract corresponds to the uploaded and publicly accessible source code. Public services like Etherscan [44] and Sourcify [45] play a crucial role in ensuring that the deployed contract matches the source code, making it easier for the community to review, validate, and trust the code's functionality. These services provide transparency by allowing users to compare the on-chain deployed bytecode with the verified source code. Furthermore, the verification process can be automated in a Continuous Deployment (CD) pipeline using tools like truffle-plugin-verify and hardhat-etherscan, which integrate with platforms like Etherscan to streamline the verification process.

In addition to contract verification, it is essential to monitor and analyze the performance, reliability, and overall health of smart contracts and their associated applications. This can be achieved through various monitoring tools and services that track key metrics and events. Public block explorers such as Etherscan, Etherchain, and Blockchain [46] serve as comprehensive analytics platforms and search engines, providing real-time data on blocks, transactions, accounts, miners, balances, and other on-chain activities across both Mainnets and Testnets. These platforms allow developers and users to track the status of smart contracts and verify the consistency of their execution on the blockchain.

For private blockchain networks, tools like BlockScout [47] and Ethernal [48] are tailored for private EVM-based networks. These explorers are designed to offer similar functionalities as public block explorers, but they cater specifically to private chains, providing insights into the transactions and events occurring within these networks.

Moreover, dedicated blockchain nodes can be implemented using advanced platforms like Tenderly [49], OpenZeppelin Sentinel [50], and Parsiq [51], which enable developers to trace smart contract activities and capture specific transaction events, blocks, and logs. These tools allow for the monitoring of state variables, tracking event emissions, and identifying deviations in gas consumption or contract execution. Such detailed tracking ensures that

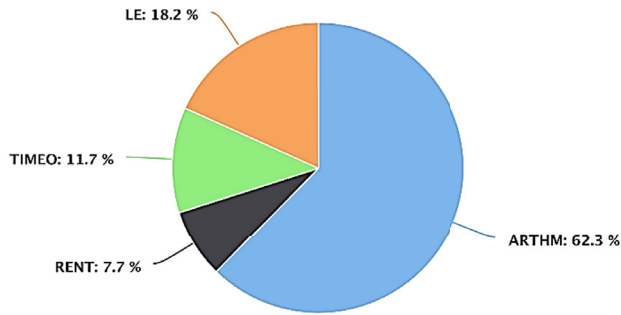


FIGURE 5. Vulnerabilities' proportion.

any unexpected behaviors or performance bottlenecks are identified and can be addressed promptly.

In addition to contract-specific monitoring, there are several infrastructure monitoring solutions that focus on network performance. Platforms like Prometheus and cAdvisor are widely used to collect and analyze statistics from network nodes. These tools offer insights into system operations by monitoring the performance of infrastructure components, such as nodes, networks, and other system resources. By leveraging these tools, developers can optimize blockchain networks and better understand the health and performance of their deployed smart contracts.

By integrating these monitoring and verification tools, blockchain projects can ensure not only the correctness of the code deployed on the blockchain but also the ongoing health and performance of the smart contracts and the overall system.

VII. EXPERIMENTAL EVALUATION

A. DATASET

In this part of the study, we assess the effectiveness of our proposed method using a publicly available dataset, analyzed from multiple perspectives. The ScrawID dataset [52], which comprises real-world Ethereum smart contracts annotated with known vulnerabilities, was selected as the foundation for our evaluation. This dataset enabled us to focus on common and critical vulnerabilities including Arithmetic (ARTHM), Block Dependency (TimeO), Token Frozen (LE), and Reentrancy (RENT). For the experimental phase, we analyzed 5,219 smart contracts, each potentially containing more than one type of vulnerability. This approach allowed us to comprehensively measure our method's capability in detecting and handling diverse security issues across a broad range of contract types and threat profiles.

B. EXPERIMENTAL ENVIRONMENT

During the experiment, GitLab CI/CD is employed to automate the manual procedures involved in building, testing, releasing, and deploying smart contracts across both the Continuous Integration (CI) and Continuous Deployment (CD) phases. This automation streamlines the DevSecOps pipeline by integrating a variety of tools that support secure, efficient,

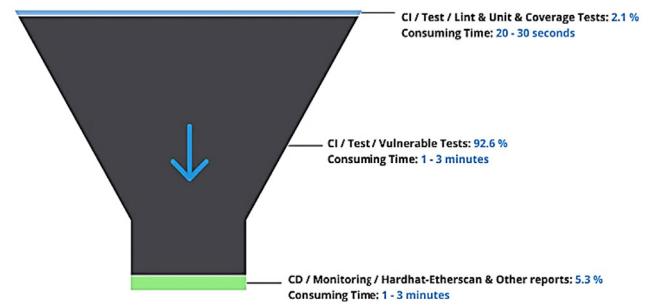


FIGURE 6. Funneling detection rates & consuming times by tests.

and reliable development workflows. These tools span from development environments like Remix IDE and Truffle Build, to testing and security tools such as Truffle Test, Solcoverage, Ganache, and vulnerability detection techniques, ensuring comprehensive code validation and quality assurance. Additionally, for TestNet and MainNet deployments, services like Goerli, Etherscan, and hardhat-etherscan are used to validate and verify contracts. In production environments, AWS Amazon Managed Blockchain supports scalable MainNet deployment and operation. Table 4 provides a detailed overview of the tools utilized within this DevSecOps framework, highlighting their respective roles in the CI/CD lifecycle.

C. ABLATION STUDY

To systematically evaluate the contribution of individual components within the DevSecOps pipeline, we conducted an ablation study based on detection rates and performance metrics across different testing stages. As depicted in Figure 6, the Continuous Integration (CI) phase utilizes standard testing techniques, linting, unit testing, and coverage analysis, which collectively detected approximately 2.1% of malicious cases. While foundational, these tests demonstrated limited capacity in identifying deeper security flaws.

In contrast, when vulnerability-specific tests were introduced during the CI phase, the detection rate surged significantly, uncovering 92.6% of vulnerabilities. These tests proved particularly effective across a range of vulnerability categories, consistently achieving over 89% absolute detection rates, as shown in Figure 7. Vulnerabilities such as Reentrancy (RENT) and Arithmetic (ARTHM) were reliably captured through automated test coverage. Moreover, residual vulnerabilities, those potentially undetected during CI were later revealed through insights derived from blockchain observability platforms such as Etherscan, as well as third-party reports, during the Continuous Deployment (CD) and monitoring phases, contributing an additional 5.3% in case detection.

To quantify the model's performance, we benchmarked it against three baseline NLP-based classification models that illustrated in Table 5:

- Model 1: distilbert-base-uncased [60]

TABLE 4. Tools using in the valuation.

Tool/Service	Environment	Phase	Description
Remix IDE [53]	Development	CI/Code	Provides a comprehensive environment for Solidity development, including contract verification, debugging, and deployment.
Truffle Build [54]	Development	CI/Build	Facilitates smart contract building and integrates seamlessly with GitLab CI/CD for automation.
GitLab CI/CD Pipeline [55]	Development TestNets MainNets	CI CD	Automates the process of building, testing, and deploying smart contracts, streamlining the development workflow.
Truffle Migration [54]	Development TestNet MainNet	CD	Automates smart contract deployment to various environments, integrated with GitLab CI/CD for smooth transitions between testing and deployment.
Truffle Test [54]	Development	CI/Test	Conducts unit tests and coverage tests for smart contracts to ensure correctness and functionality.
Sol-coverage [56]	Development	CI/Test	Provides coverage testing for smart contracts, ensuring that all code paths are adequately tested.
Ganache [57]	Development	CI/Test	Local in-memory blockchain emulator for testing and development purposes, simulating Ethereum transactions.
Vulner-ability Detection Techniques (Section 4)	Development	CI/Test	Security analysis techniques integrated into the CI pipeline, focusing on source code or bytecode inspections to prevent vulnerabilities.
Goerli [43]	TestNet	CD/Deploy	An Ethereum TestNet with PoA consensus for testing smart contracts before deployment on MainNet.
Etherscan [44]	TestNet MainNet	CD/Operate & Monitor	Blockchain explorer for exploring TestNets and MainNets; used for verifying transactions and contracts.
hardhat-etherscan [58]	TestNet MainNet	CD/Operate & Monitor	Automates contract verification on Etherscan, ensuring alignment between deployed contracts and source code.
AWS Amazon Managed Blockchain [59]	MainNet	CD/Deploy	A fully managed service for scalable blockchain networks, supporting frameworks like Ethereum for deployment and ongoing management.
AmazonQ	Development TestNets MainNets	CI CD	Amazon Q, AWS's generative AI assistant, serves as an agentic AI that enhances smart contract security across the DevSecOps lifecycle. It autonomously analyzes code for vulnerabilities like reentrancy or access control flaws, provides real-time feedback in IDEs, and integrates into CI/CD pipelines to enforce security policies. By leveraging AWS tools (e.g., Inspector, CodeGuru Security), Q can suggest secure code fixes, detect misconfigurations, and halt risky deployments. It continuously learns from past threats and user feedback to improve detection and recommendations. Amazon Q also supports compliance by mapping development activities to standards (e.g., GDPR, ISO 27001) and generating audit-ready reports. As an intelligent, adaptive agent, it accelerates vulnerability detection and remediation, enabling secure, scalable smart contract development.

- Model 2: prunebert-base-uncased-6-finepruned-w-distil-mnli [61]
- Model 3: distilbert-base-uncased-finetuned-sst-2-english [62]

Our proposed model consistently outperformed all baselines across all vulnerability types, achieving near-perfect accuracy for ARTHM (0.99) and strong results for RENT (0.97), TIMEO (0.97), and LE (0.97). These results are indicative of the effectiveness of integrating advanced scanning tools and automated security validation within the DevSecOps workflow.

This ablation study affirms that while standard tests serve as a necessary foundation in CI, the inclusion of vulnerability-specific automated testing is critical for early

detection. Furthermore, the integration of observability tools and CD-phase monitoring adds an essential safety net, capturing issues that slip past initial testing stages. Altogether, these findings validate the importance of a layered DevSecOps architecture, complementing static and dynamic analysis tools with real-time monitoring and advanced AI-powered models, to ensure robust security and minimize exploit risks in smart contract deployments.

Evidently, incorporating advanced scanning tools into the DevSecOps process can progressively enhance the detection rate of vulnerabilities by enabling earlier, more accurate identification of security flaws throughout the development lifecycle. These tools, when integrated with Continuous Integration and Continuous Deployment pipelines, allow for

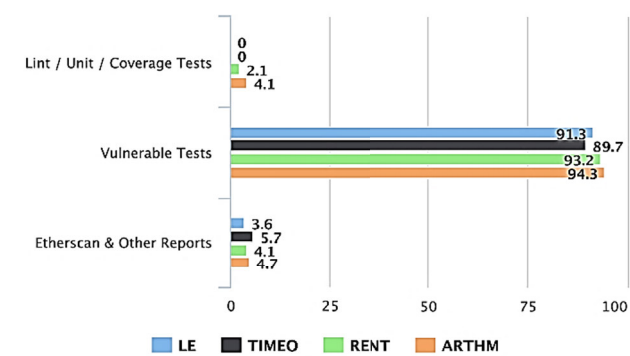


FIGURE 7. Detection rates by vulnerability types.

TABLE 5. Performance model comparison.

Vulnerability	Model 1	Model 2	Model 3	Our model
RENT	0.91	0.94	0.92	0.97
TIMEO	0.95	0.93	0.97	0.97
ARTHM	0.93	0.95	0.94	0.99
LE	0.91	0.93	0.91	0.97

automated code analysis, behavioral monitoring, and real-time validation, strengthening the system’s overall security posture and reducing the risk of vulnerabilities reaching production environments.

VIII. DISCUSSION

DevSecOps principles have seen widespread adoption across numerous software development domains, including blockchain-based applications. When integrated with appropriate tooling frameworks, DevSecOps enables developers to streamline the development and deployment pipeline while embedding robust security analysis and release management practices. However, the unique characteristics of blockchain, particularly its decentralized architecture and immutable ledger, introduce challenges not typically encountered in traditional software projects. A prominent example is the inherent difficulty in updating smart contracts post-deployment due to their immutable nature. This constraint limits the applicability of standard Continuous Delivery (CD) strategies such as canary releases, feature toggles, and A/B testing, which are commonly used in conventional software delivery. Nevertheless, updateable and modular smart contract development patterns [38] offer a promising solution, enabling seamless upgrades and bug fixes throughout the contract lifecycle.

Additionally, as machine learning (ML) becomes increasingly integrated into smart contract analysis, the threat of poisoning attacks emerges as a significant concern. These attacks aim to compromise the integrity of ML models by manipulating training data or altering model behavior, thereby causing the model to overlook real vulnerabilities or misclassify secure code as flawed. Common vectors include

training data manipulation, model parameter tampering, and adversarial inputs designed to evade detection. To counter these threats, a DevSecOps-aligned strategy should emphasize data sanitization, model robustness through techniques like adversarial training and anomaly detection, continuous retraining with secure datasets, and hybrid verification methods that combine ML with formal verification.

In response to these multifaceted challenges, this study proposes a dynamic and adaptive vulnerability detection approach that integrates security tools at every stage of the DevSecOps lifecycle. By embedding security workflows within both Continuous Integration (CI) and Continuous Delivery (CD) processes, this model promotes a shift-left security paradigm and ensures comprehensive protection across the smart contract development pipeline. Despite the availability of mature development tools supporting extensive automation and rigorous testing, challenges persist in defining essential testing standards and selecting from an ever-evolving ecosystem of tools. Addressing these issues is imperative for achieving greater security, resilience, and operational efficiency in blockchain-based applications.

IX. CONCLUSION

This research highlights the critical importance of adopting a DevSecOps framework tailored specifically for blockchain systems, particularly in addressing vulnerabilities inherent in blockchain-based applications. The proposed dynamic and adaptive approach integrates a suite of vulnerability detection tools throughout the Continuous Integration and Continuous Deployment (CI/CD) pipeline, embedding security directly into each phase of the development workflow. This integration not only enhances automation and early detection capabilities but also promotes a culture of shared responsibility among development, operations, and security teams. As a result, the approach ensures both the quality and security of smart contract development are upheld throughout its entire lifecycle.

Looking ahead, future research should emphasize the development of advanced, composite testing strategies that incorporate multiple techniques, such as static and dynamic analysis, fuzzing, formal verification, and machine learning, within the DevSecOps framework. These strategies should aim to reinforce the security posture of blockchain applications across diverse ecosystems, including public, private, and consortium blockchains. Particular attention should be paid to smart contract functionality, as well as broader blockchain operations such as consensus mechanisms, state management, and governance logic. Such integrated methodologies could form the foundation for scalable, cross-platform, and adaptable DevSecOps practices, particularly when paired with modular and upgradeable smart contract development paradigms [42]. These development models support continuous delivery (CD) pipelines more effectively by allowing secure code updates and patches without compromising the immutability and trust guarantees of blockchain.

Moreover, future studies should explore the transformative potential of quantum computing in revolutionizing vulnerability detection within smart contracts. Quantum Machine Learning (QML) [39] and agentic AI [25], [63], utilizing advanced quantum techniques such as quantum support vector machines, variational quantum circuits, and hybrid quantum-classical neural networks, offer significant advantages in detecting complex vulnerability patterns that often remain undetectable by classical methods. These quantum-powered approaches hold the promise of significantly enhancing vulnerability detection, particularly in the context of intricate and multifaceted smart contract behaviors that are difficult to model with traditional machine learning techniques. Furthermore, quantum computing can accelerate symbolic execution processes by enabling the parallel evaluation of multiple execution paths, thus dramatically reducing the computational overhead associated with traditional methods. This capability could lead to more efficient and thorough analysis of smart contracts, enhancing the identification of vulnerabilities.

In addition, quantum computing can improve formal verification methods by efficiently solving complex satisfiability problems, which are pivotal for ensuring the correctness and reliability of smart contracts. By leveraging quantum algorithms, it becomes possible to tackle verification challenges that are computationally prohibitive for classical systems, especially as smart contracts become more complex. Quantum algorithms may also assist in the identification of vulnerabilities within cryptographic primitives, addressing the growing need to secure blockchain systems against the potential risks posed by future quantum computing advances. Moreover, quantum computing plays a crucial role in testing post-quantum cryptographic schemes, which are essential for safeguarding blockchain systems in a future where quantum attacks may compromise existing cryptographic standards.

Furthermore, integrating agentic AI into DevSecOps represents a paradigm shift that could drastically enhance security, efficiency, and adaptability in software development workflows. By incorporating autonomous capabilities, agentic AI has the potential to provide real-time, intelligent decision-making and automated threat mitigation, thereby reducing human intervention in critical security processes. This shift towards intelligent automation in the DevSecOps pipeline could significantly streamline vulnerability detection, response, and remediation processes. As both quantum computing and agentic AI continue to evolve, they hold the potential to reshape the DevSecOps landscape, offering unprecedented capabilities to address emerging security challenges and driving innovation within the blockchain and software development sectors.

By advancing research in these areas, practitioners and researchers can push the boundaries of the DevSecOps paradigm, ensuring that blockchain systems are not only secure and resilient but also prepared for the evolving threat landscape of the future. These innovations will be crucial in addressing the growing complexity of blockchain

applications and ensuring that the underlying technologies remain future proof in the face of evolving computational threats.

ACKNOWLEDGMENT

The authors are extremely grateful to Ho Chi Minh City Open University, Vietnam, for supporting this research.

DISCLOSE STATEMENT

No potential conflict of interest was reported by the author(s).

REFERENCES

- [1] M. Swan, *Blockchain: Blueprint for a New Economy*. Santa Rosa, CA, USA: O'Reilly Media, 2015.
- [2] L. S. Sankar, M. Sindhu, and M. Sethumadhavan, "Survey of consensus protocols on blockchain applications," in *Proc. 4th Int. Conf. Adv. Comput. Commun. Syst. (ICACCS)*, Jan. 2017, pp. 1–5.
- [3] Z. Zheng, S. Xie, H. Dai, X. Chen, and H. Wang, "An overview of blockchain technology: Architecture, consensus, and future trends," in *Proc. IEEE Int. Congr. Big Data (BigData Congr.)*, Jun. 2017, pp. 557–564.
- [4] V. Buterin, "A next-generation smart contract and decentralized application platform," *White Paper*, vol. 3, no. 37, p. 21, 2014.
- [5] (2018). *Beautychain Integer Overflow*. [Online]. Available: <https://etherscan.io/token/0xc5d105e63711398af9bbff092d4b6769c82f793d>
- [6] (2019). *Hackers Steal \$40.7 Million in Bitcoin From Crypto Exchange Binance*. [Online]. Available: <https://www.coindesk.com/markets/2019/05/07/hackers-steal-407-million-in-bitcoin-from-crypto-exchange-binance/>
- [7] N. MacDonald and I. Head, "DevSecOps: How to seamlessly integrate security into DevOps," Gartner, Stamford, CT, USA, Tech. Rep. G00315283, 2016.
- [8] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor, "Making smart contracts smarter," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Vienna, Austria, Oct. 2016, pp. 254–269.
- [9] I. Nikolić, A. Kolluri, I. Sergey, P. Saxena, and A. Hobor, "Finding the greedy, prodigal, and suicidal contracts at scale," in *Proc. 34th Annu. Comput. Secur. Appl. Conf.*, San Juan, PR, USA, Dec. 2018, pp. 653–663.
- [10] C. F. Torres, J. Schtte, and R. State, "Osiris: Hunting for integer bugs in Ethereum smart contracts," in *Proc. 34th Annu. Comput. Secur. Appl. Conf.*, San Juan, PR, USA, Dec. 2018, pp. 664–676.
- [11] P. Tsankov, A. Dan, D. D. Cohen, A. Gervais, F. Buenzli, and M. Vechev, "Securify: Practical security analysis of smart contracts," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Toronto, ON, Canada, Oct. 2018, p. 6782.
- [12] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *Proc. IEEE/ACM 2nd Int. Workshop Emerg. Trends Softw. Eng. Blockchain (WETSEB)*, Piscataway, NJ, USA, May 2019, pp. 8–15.
- [13] B. Jiang, Y. Liu, and W. K. Chan, "ContractFuzzer: Fuzzing smart contracts for vulnerability detection," in *Proc. 33rd IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Montpellier, France, Sep. 2018, pp. 259–269.
- [14] J. He, M. Balunović, N. Ambroladze, P. Tsankov, and M. Vechev, "Learning to fuzz from symbolic execution with application to smart contracts," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Nov. 2019, pp. 531–548.
- [15] E. Hildenbrandt, M. Saxena, N. Rodrigues, X. Zhu, P. Daian, D. Guth, B. Moore, D. Park, Y. Zhang, A. Stefanescu, and G. Rosu, "KEVM: A complete formal semantics of the Ethereum virtual machine," in *Proc. IEEE 31st Comput. Secur. Found. Symp. (CSF)*, Piscataway, NJ, USA, Jul. 2018, pp. 204–217.
- [16] A. Permenev, D. Dimitrov, P. Tsankov, D. Drachler-Cohen, and M. Vechev, "VerX: Safety verification of smart contracts," in *Proc. IEEE Symp. Secur. Privacy (SP)*, Piscataway, NJ, USA, May 2020, pp. 1661–1677.
- [17] S. Tikhomirov, E. Voskresenskaya, I. Ivanitskiy, R. Takhaviev, E. Marchenko, and Y. Alexandrov, "SmartCheck: Static analysis of Ethereum smart contracts," in *Proc. IEEE/ACM 1st Int. Workshop Emerg. Trends Softw. Eng. Blockchain (WETSEB)*, Gothenburg, Sweden, May 2018, pp. 9–16.

- [18] C. Sendner, H. Chen, H. Fereidooni, L. Petzi, J. König, J. Stang, A. Dmitrienko, A.-R. Sadeghi, and F. Koushanfar, "Smarter contracts: Detecting vulnerabilities in smart contracts with deep transfer learning," in *Proc. 30th Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, San Diego, CA, USA: The Internet Society, Feb./Mar. 2023. [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/smarter-contracts-detecting-vulnerabilities-in-smart-contracts-with-deep-transfer-learning/>
- [19] R. Russell, L. Kim, L. Hamilton, T. Lazovich, J. Harer, O. Ozdemir, P. Ellingwood, and M. McConley, "Automated vulnerability detection in source code using deep representation learning," in *Proc. 17th IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Dec. 2018, pp. 757–762.
- [20] S. Li, Q. Xu, P. Hou, X. Chen, Y. Wang, H. Zhang, and G. Rong, "Exploring the challenges of developing and operating consortium blockchains: A case study," in *Proc. 24th Int. Conf. Eval. Assessment Softw. Eng.*, Apr. 2020, pp. 398–404.
- [21] R. Koul, "Blockchain oriented software testing—challenges and approaches," in *Proc. 3rd Int. Conf. Conver. Technol. (I2CT)*, Apr. 2018, pp. 1–6.
- [22] V. Yussupov, G. Falazi, U. Breitenbücher, and F. Leymann, "On the serverless nature of blockchains and smart contracts," 2020, *arXiv:2011.12729*.
- [23] M. Yilmaz, S. Taseel, and E. Tuzun, *Applying Blockchain to Improve the Integrity of the Software Development Process*, vol. 1060. Cham, Switzerland: Springer, 2019, pp. 260–271.
- [24] M. Beller and J. Hejderup, "Blockchain-based software engineering," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng., New Ideas Emerg. Results (ICSE-NIER)*, May 2019, pp. 53–56.
- [25] M. Fu, J. Pasuksmit, and C. Tantithamthavorn, "AI for DevSecOps: A landscape and future opportunities," 2024, *arXiv:2404.04839*.
- [26] A. Manikyala, H. P. Kommineni, A. R. Allam, M. Nizamuddin, and N. R. B. Sridharlakshmi, "Integrating cybersecurity best practices in DevOps pipelines for securing distributed systems," *ABC J. Adv. Res.*, vol. 12, no. 1, pp. 57–70, May 2023, doi: [10.18034/abcjar.v12i1.773](https://doi.org/10.18034/abcjar.v12i1.773).
- [27] Datadog, New York, NY, USA. (May 2023). *DevSecOps Maturity Model*. [Online]. Available: <https://www.alliances.global/wp-content/uploads/2023/05/DevSecOpsMaturityModel.pdf>
- [28] X. Ramaj, M. Sánchez-Gordón, V. Gkioulos, S. Chockalingam, and R. Colomo-Palacios, "Holding on to compliance while adopting DevSecOps: An SLR," *Electronics*, vol. 11, no. 22, p. 3707, Nov. 2022, doi: [10.3390/electronics11223707](https://doi.org/10.3390/electronics11223707).
- [29] *Dune Analytics*, 2020. [Online]. Available: <https://www.duneanalytics.com/>
- [30] *The DAO Attacked: Code Issue Leads to \$60 Million Ether Theft*. Accessed: 2016. [Online]. Available: <https://www.coindesk.com/markets/2016/06/17/the-dao-attacked-code-issue-leads-to-60-million-ether-theft/>
- [31] *Safemath*. Accessed: 2020. [Online]. Available: <https://docs.statechannels.org/contract-api/natspec/SafeMath/>
- [32] R. M. Bani-Hani, A. S. Shatnawi, and L. Al-Yahya, "Vulnerability detection and classification of Ethereum smart contracts using deep learning," *Future Internet*, vol. 16, no. 9, p. 321, Sep. 2024.
- [33] S. Kalra, S. Goel, M. Dhawan, and S. Sharma, "ZEUS: Analyzing safety of smart contracts," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, p. 112.
- [34] W. J.-W. Tann, X. J. Han, S. S. Gupta, and Y.-S. Ong, "Towards safer smart contracts: A sequence learning approach to detecting security threats," 2018, *arXiv:1811.06632*.
- [35] *ReCheck*. Accessed: Feb. 5, 2025. [Online]. Available: <https://recheck.io/blog/events/why-do-we-launch-datapond-and-what-is-it/>
- [36] B. Mueller, "A framework for bug hunting on the Ethereum blockchain," *ConsenSys/mythril*, 2017.
- [37] C. Liu, H. Liu, Z. Cao, Z. Chen, B. Chen, and B. Roscoe, "ReGuard: Finding reentrancy bugs in smart contracts," in *Proc. IEEE/ACM 40th Int. Conf. Softw. Eng., Companion (ICSE-Companion)*, May 2018, pp. 65–68.
- [38] D. Peral-García, J. Cruz-Benito, and F. J. García-Peñalvo, "Systematic literature review: Quantum machine learning and its applications," *Comput. Sci. Rev.*, vol. 51, Feb. 2024, Art. no. 100619, doi: [10.1016/j.cosrev.2024.100619](https://doi.org/10.1016/j.cosrev.2024.100619).
- [39] National Institute of Standards and Technology. *Post-Quantum Cryptography*. Accessed: Apr. 6, 2024. [Online]. Available: <https://csrc.nist.gov/projects/post-quantum-cryptography>
- [40] P. Chakraborty, R. Shahriyar, A. Iqbal, and A. Bosu, "Understanding the software development practices of blockchain development: A survey," in *Proc. 12th ACM/IEEE Int. Symp. Empirical Softw. Eng. Meas.*, Oct. 2018, pp. 1–10.
- [41] M. Wöhler and U. Zdun, "Design patterns for smart contracts in the Ethereum ecosystem," in *Proc. IEEE Int. Conf. Internet Things (iThings) IEEE Green Comput. Commun. (GreenCom) IEEE Cyber, Phys. Social Comput. (CPSCoM) IEEE Smart Data (SmartData)*, Jul. 2018, pp. 1513–1520.
- [42] P. Klinger, L. Nguyen, and F. Bodendorf, *Upgradeability Concept for Collaborative Blockchain-Based Business Process Execution Framework*, vol. 12404. Cham, Switzerland: Springer, 2020, pp. 127–141.
- [43] *Goerli Testnet*. Accessed: May 21, 2023. [Online]. Available: <https://goerli.net/>
- [44] *Block Explorer and Analytics Platform for Ethereum*. Accessed: May 21, 2023. [Online]. Available: <https://etherscan.io/>
- [45] *Sourcify: Product Sourcing Simplified*. Accessed: May 21, 2023. [Online]. Available: <https://www.sourcify.com/>
- [46] *Blockchain Explorer, Analytics, and Web Services*. Accessed: May 21, 2023. [Online]. Available: <https://blockchair.com/>
- [47] *Blockscout Explorer*. Accessed: May 21, 2023. [Online]. Available: <https://www.blockscout.com/>
- [48] *Etherscan for Your Blockchain*. Accessed: May 21, 2023. [Online]. Available: <https://tryethernal.com/>
- [49] *Ethereum Development Platform*. Accessed: May 21, 2023. [Online]. Available: <https://tenderly.co/>
- [50] *Sentinel*. Accessed: May 21, 2023. [Online]. Available: <https://docs.openzeppelin.com/defender/v1/sentinel>
- [51] *Parsiq*. Accessed: May 21, 2023. [Online]. Available: <https://parsiq.net/>
- [52] C. S. Yashavant, S. Kumar, and A. Karkare, "ScrawlID: A dataset of real world Ethereum smart contracts labelled with vulnerabilities," 2022, *arXiv:2202.11409*.
- [53] *Remix IDE*. Accessed: Dec. 12, 2023. [Online]. Available: <https://remix.ethereum.org>
- [54] *Truffle*. Accessed: Jan. 5, 2024. [Online]. Available: <https://archive.trufflesuite.com/truffle/>
- [55] *GitLab CI/CD*. Accessed: Jan. 17, 2024. [Online]. Available: <https://docs.gitlab.com/ee/ci/>
- [56] *Sol-Coverage*. Accessed: Feb. 5, 2024. [Online]. Available: <https://github.com/sc-forks/solidity-coverage>
- [57] *Ganache*. Accessed: Jan. 3, 2024. [Online]. Available: <https://archive.trufflesuite.com/ganache/>
- [58] *Hardhat-Verify*. Accessed: Feb. 5, 2024. [Online]. Available: <https://hardhat.org/hardhat-runner/plugins/nomicfoundation-hardhat-verify>
- [59] *Amazon Managed Blockchain*. Accessed: Feb. 5, 2024. [Online]. Available: <https://aws.amazon.com/managed-blockchain/>
- [60] *Distilbert-Base-Uncased*. Accessed: May 21, 2024. [Online]. Available: <https://huggingface.co/distilbert/distilbert-base-cased>
- [61] *Prunebert-Base-Uncased-6-Finepruned-W-Distil-MNLI*. Accessed: Mar. 22, 2024. [Online]. Available: <https://huggingface.co/huggingface/prunebert-base-uncased-6-finepruned-w-distil-mnli>
- [62] *Distilbert-Base-Uncased-Finetuned-SST-2-English*. Accessed: Apr. 3, 2024. [Online]. Available: <https://huggingface.co/distilbert/distilbert-base-uncased-finetuned-sst-2-english>
- [63] G. Pillala, D. Azarpazhooh, and S. Baxter, "DevSecOps sentinel: GenAI-driven agentic workflows for comprehensive supply chain security," *Comput. Inf. Sci.*, vol. 18, no. 1, p. 39, 2024.



NGHIA DINH received the master's degree in software engineering from the University of Bordeaux, France, and the Ph.D. degree from Ostrava Technical University, Czech Republic. He is currently a Software Architecture Enthusiast and a Computer Scientist. He has contributed to the success of many open sources and technology companies.



VINH TRUONG HOANG received the master's degree from the University of Montpellier, in 2009, and the Ph.D. degree in computer science from the University of the Littoral Opal Coast, France. He is currently an Assistant Professor and the Head of the Image Processing and Computer Graphics Department, Ho Chi Minh City Open University, Vietnam. His research interests include image analysis and feature selection.



HA DUONG THI HONG is currently a Lecturer with Ho Chi Minh City Open University. Her research interests include machine learning and feature selection.



BAY NGUYEN VAN is currently a Lecturer with Ho Chi Minh City Open University. His research interests include machine learning and OCR.



HAU NGUYEN TRUNG received the master's degree from the University of Science, Vietnam. He is currently a Lecturer with Ho Chi Minh City Open University. His research interests include machine learning and computer vision.



THIEN HO HUONG is currently a Lecturer with Ho Chi Minh City Open University. His research interests include machine learning and text mining.



KIET TRAN TRUNG received the master's degree from Ho Chi Minh City Pedagogical University, Vietnam. He is currently a Lecturer with Ho Chi Minh City Open University. His research interests include machine learning and computer vision.

...